

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



LÊ TRẦN MINH PHÚC

**TỐI ƯU CHẤT LƯỢNG - ĐA DẠNG CÓ ĐIỀU KIỆN NGŨ
NGHĨA TRONG KHÔNG GIAN TIỀM ẨN ĐỂ TẠO SINH
MÀN CHƠI THE LEGEND OF ZELDA**

**KHÓA LUẬN TỐT NGHIỆP
NGÀNH KHOA HỌC MÁY TÍNH**

TP. HỒ CHÍ MINH, 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



LÊ TRẦN MINH PHÚC

**TỐI ƯU CHẤT LƯỢNG - ĐA DẠNG CÓ ĐIỀU KIỆN NGŨ
NGHĨA TRONG KHÔNG GIAN TIỀM ẨN ĐỂ TẠO SINH
MÀN CHƠI THE LEGEND OF ZELDA**

Chuyên ngành: Khoa học máy tính
Mã số sinh viên: 2251010074 Lớp: DH22CS01C

**KHÓA LUẬN TỐT NGHIỆP
NGÀNH KHOA HỌC MÁY TÍNH**

Giảng viên hướng dẫn: **TS. Lê Đăng Giáp**

TP. HỒ CHÍ MINH, 2026

TRƯỜNG ĐẠI HỌC MỞ
THÀNH PHỐ HỒ CHÍ MINH
KHOA Đào Tạo Đặc Biệt

CỘNG HÒA XÃ HỘI CHỦ NGHĨA
VIỆT NAM

Độc lập - Tự do - Hạnh phúc

Thành phố Hồ Chí Minh, ngày tháng
..... năm 2026

GIẤY XÁC NHẬN

Tôi tên là: Lê Trần Minh Phúc

Ngày sinh: 18/04/2004

Nơi sinh: Tây Ninh

Chuyên ngành: Khoa học máy tính

Mã sinh viên: 2251010074

Tôi đồng ý cung cấp toàn văn thông tin khóa luận tốt nghiệp hợp lệ và bản quyền cho Thư viện Trường Đại học Mở Thành phố Hồ Chí Minh. Thư viện Trường Đại học Mở Thành phố Hồ Chí Minh được phép nộp toàn văn thông tin khóa luận vào hệ thống thông tin khoa học theo quy định hiện hành.

Ký tên

(Ghi rõ họ và tên)

Lê Trần Minh Phúc

Ý KIẾN CHO PHÉP BẢO VỆ KHÓA LUẬN TỐT NGHIỆP

Giảng viên hướng dẫn: TS. Lê Đăng Giáp

Sinh viên thực hiện: Lê Trần Minh Phúc

Lớp: DH22CS01C

Ngày sinh: 18/04/2004

Nơi sinh: Tây Ninh

Tên khóa luận: Tối ưu chất lượng - đa dạng có điều kiện ngữ nghĩa trong không gian tiềm ẩn để tạo sinh màn chơi The Legend of Zelda

Ý kiến của giảng viên hướng dẫn về việc cho phép sinh viên được bảo vệ khóa luận tốt nghiệp trước Hội đồng:

Thành phố Hồ Chí Minh, ngày tháng năm 2026

Giảng viên hướng dẫn

TS. Lê Đăng Giáp

LỜI CẢM ƠN

Em xin chân thành cảm ơn TS. Lê Đăng Giáp đã tận tình hướng dẫn, góp ý và đồng hành cùng em trong suốt quá trình thực hiện khóa luận.

Em xin gửi lời cảm ơn đến TS. Sanggyu Nam vì những trao đổi chuyên môn và góp ý quý báu, giúp em mở rộng góc nhìn khi triển khai đề tài. Những chỉ dẫn của thầy giúp em định hướng rõ hơn cách tiếp cận đề tài và hoàn thiện nội dung nghiên cứu.

Em cũng xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ và tạo điều kiện để em yên tâm học tập, nghiên cứu và hoàn thành khóa luận này.

Do thời gian và kinh nghiệm còn hạn chế, khóa luận chắc chắn vẫn còn thiếu sót. Em rất mong nhận được những góp ý của người đọc và các bạn để em có thể hoàn thiện hơn trong những nghiên cứu tiếp theo.

Trân trọng,

Lê Trần Minh Phúc

Tóm tắt

Khóa luận này nghiên cứu bài toán tạo sinh dungeon cho *The Legend of Zelda* theo hướng kết hợp mô hình học máy với các ràng buộc ký hiệu nhằm bảo đảm tiến trình chơi và tính hợp lệ của cấu trúc. Thay vì sinh trực tiếp toàn bộ bản đồ, hệ thống được tổ chức theo hướng tách tầng: trước hết sinh cấu trúc đồ thị nhiệm vụ, sau đó sinh chi tiết từng phòng trong không gian tiềm ẩn rời rạc. Kiến trúc đề xuất gồm bảy khối chính: bộ xử lý dữ liệu, bộ sinh cấu trúc đồ thị nhiệm vụ bằng tiến hóa, VQ-VAE ngữ nghĩa, bộ mã hóa điều kiện hai luồng, mô hình diffusion trong latent space, mô-đun hướng dẫn logic, bộ sửa lỗi ký hiệu và lớp đánh giá quality-diversity.

Cách tổ chức này cho phép mô hình duy trì quan hệ khóa – chìa, kết nối phòng và khả năng chơi được, đồng thời vẫn giữ được độ đa dạng của bố cục phòng. Hệ thống được kiểm chứng bằng các thí nghiệm ablation, so sánh theo matched-budget và các kịch bản thay đổi quy mô đồ thị nhiệm vụ để đánh giá mức độ ổn định của từng thành phần. Kết quả thực nghiệm cho thấy kiến trúc lai giúp cân bằng tốt hơn giữa tính hợp lệ, khả năng giải được và độ đa dạng nội dung so với các cấu hình không tách rõ vai trò giữa tầng đồ thị nhiệm vụ và tầng sinh phòng.

Từ đó, khóa luận khẳng định tính khả thi của hướng tiếp cận graph-first kết hợp latent diffusion và sửa lỗi ký hiệu cho bài toán tạo sinh dungeon Zelda trong bối cảnh dữ liệu hạn chế.

Từ khóa: sinh nội dung thủ tục, dungeon Zelda, graph-first, VQ-VAE, latent diffusion, sửa lỗi ký hiệu, quality-diversity.

Abstract

This thesis investigates dungeon generation for *The Legend of Zelda* through a hybrid neural-symbolic framework that separates mission-graph planning from room synthesis. Rather than generating an entire level in a single step, the proposed system first constructs a mission graph that encodes progression, locks, branches, and target structure, and then generates each room in a semantic latent space before applying symbolic post-processing to repair residual inconsistencies. The architecture combines seven components: data preprocessing, evolutionary graph search, a semantic VQ-VAE, a dual-stream condition encoder, latent diffusion, logic guidance, symbolic repair, and a quality-diversity evaluation layer.

This factorization is designed to preserve solvability and structural validity while still allowing diverse room layouts and controllable game progression. The thesis evaluates the system through fixed-seed ablations, matched-budget comparisons, and graph-scale tests in order to isolate the contribution of each block. The experimental protocol focuses on structural correctness, playability, robustness, and behavioral diversity rather than on visual fidelity alone.

Overall, the results indicate that the hybrid design provides a practical compromise between global control and local generative flexibility for Zelda-style dungeon generation. More broadly, the work shows that combining graph-first planning, latent generative modeling, and symbolic repair is a viable research direction for procedural content generation under limited data.

Keywords: procedural content generation, Zelda dungeon generation, graph-first design, VQ-VAE, latent diffusion, symbolic repair, quality-diversity.

Mục lục

Tóm tắt	
Abstract	
Danh sách hình	vi
Danh sách bảng	vii
Danh sách từ viết tắt	viii
Danh mục thuật ngữ chuyên ngành	ix
1 Giới thiệu	1
1.1 Lý do chọn đề tài	1
1.2 Mục tiêu nghiên cứu	2
1.3 Đối tượng và phạm vi nghiên cứu	2
1.4 Phương pháp nghiên cứu	3
1.5 Câu hỏi nghiên cứu	3
1.6 Đóng góp của luận văn	4
1.7 Bố cục luận văn	4
2 Tổng quan nghiên cứu và cơ sở lý thuyết	5
2.1 Nền tảng lý thuyết	5
2.1.1 Sinh nội dung thủ tục trong trò chơi	5
2.1.2 Biểu diễn rời rạc và sinh trong không gian tiềm ẩn	6
2.1.3 Điều kiện hóa theo đồ thị nhiệm vụ	7
2.1.4 Khung khái niệm và chuẩn hóa thuật ngữ	7
2.1.5 Tiêu chí phân tích và đánh giá	7

2.2	Tổng quan công trình liên quan	8
2.2.1	Hướng đồ thị nhiệm vụ và PCG dựa trên tìm kiếm	8
2.2.2	Thuật toán tìm kiếm để kiểm tra và giải dungeon	9
2.2.3	Hướng sinh đồ thị học sâu và diffusion rời rạc	9
2.2.4	Hướng PCGML ở cấp độ phòng	10
2.2.5	Hướng tăng tốc suy luận và sinh rời rạc theo mã	10
2.2.6	Hướng lai nơ-ron – ký hiệu	11
2.3	Khoảng trống nghiên cứu	11
2.4	Kết luận chương	12
3	Phương pháp đề xuất	13
3.1	Mục tiêu và phạm vi chương	13
3.2	Kiến trúc tổng thể và hợp đồng dữ liệu	14
3.2.1	Phân biệt đồ thị kiến trúc và đồ thị nhiệm vụ	14
3.2.2	Mô hình đồ thị kiến trúc 7 khối	15
3.2.3	Ma trận hợp đồng dữ liệu theo khối	16
3.2.4	Dòng chảy đầu-cuối của quy trình	18
3.2.5	Biểu diễn dữ liệu và schema đầu vào	19
3.2.6	Tiền xử lý dữ liệu và chính sách chia tập	19
3.2.7	Ràng buộc triển khai và bất biến dữ liệu	20
3.3	Thiết kế chi tiết các khối trong pipeline	21
3.3.1	Nguyên tắc lựa chọn thiết kế giữa các phương án thay thế	21
3.3.2	Mô hình sinh đồ thị nhiệm vụ (Khối I): cấu trúc và tiến trình	22
3.3.3	Khối mã hóa điều kiện graph-to-room (Khối III)	24
3.3.4	Khối sinh nơ-ron: VQ-VAE (Khối II) và khuếch tán tiềm ẩn (Khối IV)	25
3.3.5	Khối hướng dẫn logic và sửa lỗi ký hiệu (Khối V–VI)	25

3.3.6	Khối xác nhận hậu sinh: oracle cứng, P-CBS và giao thức bàn giao validation (Khối VII)	28
3.4	Thuật toán huấn luyện và suy luận trong mã nguồn	31
3.4.1	Huấn luyện theo giai đoạn	31
3.4.2	Suy luận nhiều phòng và cơ chế fallback	33
3.4.3	Đánh giá sau sinh và chỉ số vận hành	39
3.4.4	Giả định vận hành, chế độ lỗi và biện pháp giảm thiểu	40
3.5	Cấu hình triển khai và khả năng tái lập	42
3.6	Ánh xạ nội dung phương pháp vào luồng hiện thực	43
3.7	Liên kết với thiết kế thực nghiệm Chương 4	44
3.8	Tóm tắt chương	44
4	Kết quả thực nghiệm và thảo luận	45
4.1	Mục tiêu kiểm chứng và câu hỏi thực nghiệm	45
4.2	Thiết kế thực nghiệm	46
4.2.1	Thiết lập tái lập và môi trường thực nghiệm	46
4.2.2	Dữ liệu và kịch bản đánh giá	46
4.2.3	Baseline và cấu hình so sánh	47
4.2.4	Chỉ số đánh giá và suy luận thống kê	47
4.2.5	Giao thức thí nghiệm	47
4.3	Kết quả định lượng	48
4.3.1	Kết quả huấn luyện VQ-VAE nền tảng	48
4.3.2	Kết quả tổng thể so với baseline	49
4.3.3	Kết quả ablation theo khối pipeline	50
4.3.4	Kết quả matched-budget ở tầng đồ thị nhiệm vụ	50
4.3.5	Kết quả OOD theo quy mô dungeon	51
4.3.6	Độ tin cậy vận hành và chi phí	51
4.4	Thảo luận	52

4.4.1	Giải thích cơ chế cải thiện	52
4.4.2	Đánh đổi chất lượng, đa dạng và chi phí	52
4.4.3	Mối đe dọa tính hiệu lực và giới hạn	52
4.5	Tóm tắt chương	53
5	Kết luận và hướng phát triển	54
5.1	Tổng kết đóng góp	54
5.2	Hạn chế của nghiên cứu	55
5.3	Hướng phát triển	55
	PHỤ LỤC	57
A	Khung ký hiệu và mô hình chi phí	57
A.1	Ký hiệu sử dụng trong phụ lục	57
A.2	Mô hình chi phí chuẩn hóa	57
B	Thuật toán hỗ trợ trong phụ lục	57
B.1	Tiền xử lý dữ liệu và chia tập theo nguồn	58
B.2	Kiểm tra hợp lệ và khả giải dungeon	58
B.3	Sửa phòng có phản hồi ký hiệu	58
B.4	Lập lịch sinh phòng theo lớp phụ thuộc	58
B.5	Sinh một phòng với cơ chế quay về mô hình giáo viên	61
B.6	Đánh giá dungeon sau ghép	61
C	Chứng minh độ phức tạp	62
C.1	Bổ đề phân lớp topo	62
C.2	Mệnh đề cho sinh cấu trúc đồ thị tiến hóa	62
C.3	Mệnh đề cho lập lịch nhiều phòng	63
C.4	Mệnh đề cho huấn luyện một epoch diffusion	63
C.5	Mệnh đề cho suy luận một phòng với cơ chế quay về mô hình giáo viên	64

C.6	Mệnh đề cho đánh giá hậu kỳ sau ghép dungeon	64
D	Phép tính minh họa cho độ phức tạp thực toán	64
D.1	Nguyên tắc tính minh họa	64
D.2	Ví dụ 1: chi phí sửa một phòng	65
D.3	Ví dụ 2: chi phí lập lịch nhiều phòng	65
D.4	Ví dụ 3: ngân sách đánh giá ở tầng đồ thị nhiệm vụ	65
D.5	Ví dụ 4: kỳ vọng chi phí khi có cơ chế quay về mô hình giáo viên	66
D.6	Kiểm tra định tính nhánh latent liên tục	66

Danh sách hình

3.1	Kiến trúc 7 khối của quy trình sinh nơ-ron–ký hiệu theo hướng graph-first. Mũi tên nét đứt biểu diễn các nhánh hướng dẫn logic và đánh giá tùy cấu hình.	16
3.2	Luồng nội bộ của quy trình sinh phòng, bao gồm bước hiệu chỉnh ký hiệu và cơ chế quay về mô hình giáo viên.	34
3.3	Luồng lập lịch sinh phòng theo lớp phụ thuộc và nhóm latent.	34

Danh sách bảng

3.1	Ký hiệu dùng xuyên suốt Chương 3	14
3.2	Ma trận hợp đồng dữ liệu và cơ chế an toàn theo từng khối	17
3.3	Các chế độ lỗi chính và cơ chế giảm thiểu trong đường chạy Chương 3	42
4.1	So sánh sáu cấu hình VQ-VAE trên cùng lịch huấn luyện 300 epoch	49

Danh sách từ viết tắt

Từ viết tắt	Ý nghĩa
AI	Artificial Intelligence
CFG	Classifier-Free Guidance
CMA-ME	Covariance Matrix Adaptation MAP-Elites
CVT	Centroidal Voronoi Tessellation
CVT-MAP-Elites	Centroidal Voronoi Tessellation MAP-Elites
DDIM	Denoising Diffusion Implicit Model
DDPM	Denoising Diffusion Probabilistic Model
GNN	Graph Neural Network
GPS	Graph Positioning System (Graph Transformer Recipe)
LCM	Latent Consistency Model
LDM	Latent Diffusion Model
MAP-Elites	Multi-dimensional Archive of Phenotypic Elites
PCG	Procedural Content Generation (Sinh nội dung thủ tục)
PCGML	Procedural Content Generation via Machine Learning
QD	Quality-Diversity
RQ	Research Question
VGLC	Video Game Level Corpus
VQ-VAE	Vector Quantized Variational Autoencoder
WFC	Wave Function Collapse

Danh mục thuật ngữ chuyên ngành

Thuật ngữ	Định nghĩa ngắn gọn
Graph-first generation	Chiến lược sinh theo hai bước: (1) sinh đồ thị nhiệm vụ và ràng buộc tiến trình trước; (2) sinh bố cục/hình học phòng để hiện thực hóa cấu trúc đó.
Mission graph	Đồ thị mô tả các phòng/nút nhiệm vụ và quan hệ kết nối, điều kiện mở khóa trong dungeon.
Room layout	Bố cục tile bên trong một phòng, thể hiện tường, cửa, vật cản, thực thể và đường đi.
Latent space	Không gian biểu diễn nén dùng để mã hóa đặc trưng ngữ nghĩa của phòng trước khi sinh mẫu mới.
Discrete latent code	Mã biểu diễn rời rạc trong latent space, thường được học bởi VQ-VAE cho dữ liệu tile.
Conditional diffusion	Mô hình diffusion sinh mẫu dưới điều kiện đầu vào (ví dụ: ngữ cảnh phòng, ràng buộc đồ thị).
Graph conditioning	Cơ chế đưa thông tin từ mission graph vào mô hình sinh để giữ nhất quán tiến trình toàn cục.
Semantic constraint	Ràng buộc ngữ nghĩa gameplay, ví dụ tính hợp lệ khóa – chìa, boss room hoặc thứ tự mở cửa.
Logic guidance	Thành phần hướng dẫn quá trình sinh dựa trên luật và tín hiệu logic để giảm mẫu vi phạm.
Symbolic repair	Bước hậu xử lý dùng luật ký hiệu để sửa các lỗi cấu trúc hoặc tiến trình còn sót lại.
Solvability	Mức độ mà dungeon có thể được hoàn thành theo luật chơi và trạng thái vật phẩm hợp lệ.
Validity	Mức độ tuân thủ của dungeon đối với các ràng buộc hình học, kết nối và tiến trình nhiệm vụ.
Quality-Diversity (QD)	Nhóm phương pháp tối ưu đồng thời chất lượng nghiệm và độ đa dạng của nghiệm trong không gian hành vi.
Behavior space	Không gian do các bộ mô tả xác định, dùng để phân bố và so sánh nghiệm trong bài toán QD.

Behavior descriptor	Bộ mô tả hành vi của một nghiệm; thường là vector đặc trưng dùng để định vị nghiệm trong behavior space.
Archive	Kho lưu trữ QD giữ lại nghiệm tốt nhất ở mỗi ô của behavior space.
Elite	Nghiệm tốt nhất đang đại diện cho một ô trong archive.
MAP-Elites	Thuật toán QD lấp đầy archive bằng elite tốt nhất cho từng ô descriptor.
Coverage	Tỷ lệ ô archive đã có ít nhất một elite.
QD-score	Chỉ số tổng hợp phản ánh đồng thời chất lượng nghiệm và mức độ phủ archive.
CVT archive	Archive dựa trên Centroidal Voronoi Tessellation để chia behavior space thành các ô gần đều.
CVT-emitter	Cơ chế phát sinh nghiệm mới dựa trên CVT archive, dùng các elite hiện có để dẫn hướng khám phá.
Search-based PCG	Nhóm phương pháp dùng thuật toán tìm kiếm để tối ưu hoặc khám phá không gian nội dung ứng viên.
State-space search	Tìm kiếm trên không gian trạng thái chơi để kiểm tra đường đi, khả giải và thứ tự mở khóa.
Constraint solving	Bài toán tìm nghiệm thỏa toàn bộ ràng buộc hình học và ràng buộc lối chơi.
Ablation study	Thiết kế thực nghiệm tắt/bật từng thành phần để định lượng đóng góp của mỗi mô-đun.
Reproducibility	Khả năng tái lập kết quả thực nghiệm dưới cùng cấu hình, seed và dữ liệu đầu vào.

CHƯƠNG 1

Giới thiệu

Chương này mở đầu luận văn bằng cách trình bày bối cảnh, động cơ và mục tiêu nghiên cứu. Nội dung chương cũng xác định phạm vi, câu hỏi nghiên cứu và các đóng góp chính, làm cơ sở cho phần nền tảng lý thuyết và phương pháp ở các chương tiếp theo.

1.1 Lý do chọn đề tài

Trong phát triển trò chơi hiện đại, nhu cầu mở rộng nội dung với chi phí hợp lý khiến sinh nội dung thủ tục (Procedural Content Generation – PCG) trở thành một hướng nghiên cứu có ý nghĩa thực tiễn rõ rệt [1]. Tuy nhiên, với bài toán dungeon, giá trị của hệ tạo sinh không chỉ nằm ở số lượng mẫu tạo ra mà còn ở mức độ bảo toàn tiến trình nhiệm vụ và khả năng chơi được [2, 3, 4].

Đối với *The Legend of Zelda*, dungeon là một cấu trúc nhiệm vụ có logic nội tại, trong đó quan hệ giữa cửa, khóa, chìa và mục tiêu phải nhất quán từ đầu đến cuối. Nếu hình thái phòng tốt về bề mặt nhưng lệch khỏi đồ thị nhiệm vụ, người chơi có thể rơi vào bế tắc hoặc đi lệch thiết kế. Các tiếp cận tiến hóa trong miền này cho thấy dungeon có thể chơi được, nhưng bài toán cân bằng giữa tính đúng đắn tiến trình, chất lượng bố cục và độ đa dạng vẫn còn mở [3]. Trong khi đó, các tiếp cận học máy cho cấu trúc có điều kiện thường mạnh ở học mẫu cục bộ nhưng vẫn cần cơ chế điều kiện hóa hoặc hậu kiểm để giữ ràng buộc toàn cục [5, 6, 7, 8, 9]. Khoảng trống này là lý do trực tiếp để luận văn lựa chọn kiến trúc lai nơ-ron – ký hiệu nhằm dung hòa năng lực tạo mẫu và năng lực kiểm soát.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát của luận văn là xây dựng và kiểm chứng một hệ thống tạo sinh dungeon Zelda có khả năng cân bằng ba yêu cầu thường xung đột nhau: tính hợp lệ của lối chơi, chất lượng cấu trúc phòng và độ đa dạng nội dung trong điều kiện dữ liệu huấn luyện giới hạn.

Để đạt mục tiêu trên, luận văn tổ chức pipeline theo nguyên tắc ưu tiên cấu trúc nhiệm vụ: xác lập đồ thị nhiệm vụ trước, sau đó mới sinh chi tiết phòng. Trên nền đồ thị này, hệ thống phát triển khối sinh phòng trong không gian tiềm ẩn rời rạc bằng VQ-VAE kết hợp diffusion có điều kiện nhằm tăng độ ổn định của mẫu sinh và giữ ngữ nghĩa ở mức tile [10, 11, 12]. Song song với khối sinh, kiến trúc tích hợp cơ chế hướng dẫn logic và sửa lỗi ký hiệu để giảm sai lệch về đường đi, quan hệ khóa – chìa và thứ tự tiến trình. Toàn bộ hệ được đánh giá bằng thực nghiệm đối chứng theo hướng phân tích đóng góp thành phần để làm rõ vai trò từng khối và mức đánh đổi giữa chất lượng với độ đa dạng. Bên cạnh đó, nghiên cứu đặt mục tiêu so sánh công bằng giữa các phương pháp sinh đồ thị nhiệm vụ dưới cùng ngân sách tính toán, đồng thời kiểm tra khả năng khái quát trong và ngoài phân phối khi quy mô phòng thay đổi. Kỳ vọng thực nghiệm là tăng tỉ lệ dungeon hợp lệ, duy trì khả năng giải ở mức cao và hạn chế hiện tượng lặp mẫu.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của luận văn là bài toán tạo sinh dungeon Zelda từ đồ thị nhiệm vụ và bố cục phòng dạng lưới tile. Phạm vi nghiên cứu được giới hạn trong miền dữ liệu Zelda cùng các ràng buộc lối chơi cốt lõi, bao gồm kết nối phòng, quan hệ khóa – chìa và điều kiện hoàn thành nhiệm vụ.

Về mặt kỹ thuật, luận văn tập trung vào chuỗi xử lý lai nơ-ron – ký hiệu gồm bốn lớp chính: sinh cấu trúc đồ thị nhiệm vụ, sinh phòng có điều kiện ngữ nghĩa trong không gian tiềm ẩn, kiểm tra hậu kỳ logic và sửa lỗi ký hiệu. Về mặt đánh giá, nghiên cứu ưu

tiên các chỉ số định lượng liên quan đến tính hợp lệ, khả năng giải được, chất lượng và đa dạng. Luận văn không triển khai khảo sát người dùng quy mô lớn; thay vào đó, nghiên cứu chuẩn hóa bộ dữ liệu đánh giá mù để phục vụ bước đánh giá người dùng ở giai đoạn mở rộng.

1.4 Phương pháp nghiên cứu

Luận văn áp dụng phương pháp nghiên cứu hỗn hợp, kết hợp tổng quan tài liệu, thiết kế hệ thống và thực nghiệm định lượng. Ở giai đoạn đầu, các hướng nghiên cứu then chốt trong PCG, PCGML, sinh đồ thị nhiệm vụ, diffusion trong không gian tiềm ẩn và hệ lai nơ-ron – ký hiệu được hệ thống hóa để xác lập nền tảng lý thuyết và nhận diện khoảng trống nghiên cứu [1, 5, 13].

Từ nền tảng đó, kiến trúc được thiết kế theo hướng mô-đun để tách rõ vai trò từng khối và giảm phụ thuộc chéo, qua đó thuận lợi cho kiểm chứng thành phần. Hệ thống sau thiết kế được hiện thực thành pipeline đầu-cuối trên mã nguồn luận văn với cấu hình kiểm soát nhằm bảo đảm khả năng tái lập. Về đánh giá, nghiên cứu sử dụng giao thức cố định hạt giống ngẫu nhiên, so sánh dưới cùng ngân sách tính toán và phân tích ngoài phân phối để kiểm tra độ ổn định khi quy mô đồ thị nhiệm vụ thay đổi. Về suy luận thống kê, luận văn dùng bootstrap theo cặp để ước lượng khoảng tin cậy 95% [14], kiểm định hoán vị dấu theo cặp để ước lượng giá trị p , và hiệu chỉnh đa kiểm định theo Benjamini-Hochberg [15]. Cuối cùng, các kết quả được phân tích theo nhóm chỉ số hợp lệ, giải được, chất lượng và đa dạng để rút ra kết luận thực chứng.

1.5 Câu hỏi nghiên cứu

Từ mục tiêu và phạm vi đã xác định, luận văn tập trung vào bốn câu hỏi định lượng cốt lõi. (1) Chiến lược ưu tiên cấu trúc nhiệm vụ có giúp cải thiện đồng thời tính hợp lệ gameplay và chất lượng tạo phòng so với cấu hình không tách tầng hay không? (2)

Thành phần nào đóng góp chi phối trong pipeline đề xuất, gồm sinh cấu trúc đồ thị nhiệm vụ, VQ-VAE, diffusion có điều kiện theo đồ thị, hướng dẫn logic và sửa lỗi ký hiệu? (3) Mức đóng góp thực tế của hậu kiểm tra và sửa lỗi ký hiệu đối với độ tin cậy của dungeon trong bối cảnh dữ liệu nhỏ là bao nhiêu? (4) Lợi thế của phương pháp đề xuất có còn được duy trì khi so sánh dưới cùng ngân sách tính toán với các môc sinh đồ thị nhiệm vụ và khi chuyển sang các trường hợp ngoài phân phối theo số lượng phòng hay không?

1.6 Đóng góp của luận văn

Đóng góp của luận văn trải trên ba phương diện: kiến trúc, hiện thực hệ thống và đánh giá thực nghiệm. Ở phương diện kiến trúc, nghiên cứu đề xuất mô hình lai nơ-ron – ký hiệu có phân tầng rõ giữa lớp nhiệm vụ và lớp tạo phòng, qua đó tăng khả năng kiểm soát tiến trình lối chơi. Ở phương diện hiện thực, luận văn xây dựng khối sinh phòng trong không gian tiềm ẩn rời rạc có điều kiện ngữ nghĩa, đồng thời tích hợp hướng dẫn logic và sửa lỗi ký hiệu để nâng tỉ lệ dungeon hợp lệ, giải được. Ở phương diện thực nghiệm, nghiên cứu thiết kế giao thức gồm phân tích đóng góp thành phần với hạt giống cố định và kiểm định thống kê, so sánh dưới cùng ngân sách tính toán giữa các phương pháp sinh đồ thị nhiệm vụ, đánh giá ngoài phân phối theo quy mô phòng, và chuẩn hóa bộ dữ liệu đánh giá mù cho bước đánh giá người dùng mở rộng.

1.7 Bố cục luận văn

Phần còn lại của khóa luận được tổ chức thành bốn chương. Chương 2 trình bày tổng quan nghiên cứu và cơ sở lý thuyết, đồng thời xác định khoảng trống mà đề tài hướng tới. Chương 3 mô tả chi tiết kiến trúc đề xuất, luồng dữ liệu và cách hiện thực các mô-đun trong hệ thống. Chương 4 trình bày thiết kế thực nghiệm, giao thức đánh giá, kết quả định lượng và phần thảo luận. Chương 5 tổng kết các đóng góp chính, nêu hạn chế và đề xuất hướng phát triển tiếp theo.

CHƯƠNG 2

Tổng quan nghiên cứu và cơ sở lý thuyết

Chương này hệ thống hóa các nền tảng lý thuyết và các hướng nghiên cứu liên quan trực tiếp đến bài toán tạo sinh dungeon Zelda. Cách trình bày đi từ các khái niệm nền tảng của PCG và mô hình sinh rời rạc, sang các hướng nghiên cứu về điều kiện hóa theo đồ thị, rồi tổng hợp các công trình liên quan và khoảng trống nghiên cứu làm cơ sở cho kiến trúc đề xuất ở Chương 3.

2.1 Nền tảng lý thuyết

Phần này tổng hợp các nền tảng cốt lõi làm cơ sở cho kiến trúc đề xuất, từ PCG truyền thống đến các mô hình sinh trong không gian tiềm ẩn. Cách trình bày đi từ bức tranh tổng quát đến các cơ chế cụ thể đang được hiện thực trong hệ thống.

2.1.1 Sinh nội dung thủ tục trong trò chơi

Trong nghiên cứu sinh nội dung thủ tục (PCG), ba hướng tiếp cận được dùng nhiều nhất là dựa trên luật, dựa trên tìm kiếm và dựa trên học máy tạo sinh [1, 5, 16]. Mỗi hướng có thể mạnh riêng: phương pháp dựa trên luật thuận lợi cho kiểm soát ràng buộc, phương pháp tìm kiếm phù hợp với tối ưu đa mục tiêu, còn phương pháp học máy mạnh ở khả năng học mẫu không gian phức tạp từ dữ liệu.

Ở nhánh học máy cho PCG, tổng quan PCGML cho thấy bài toán không chỉ nằm ở sinh mẫu mới mà còn ở khả năng sửa lỗi, phân tích nội dung và làm việc trong điều kiện dữ liệu nhỏ [4]. Nhận định này đặc biệt phù hợp với miền Zelda, nơi ràng buộc lối chơi khiến chất lượng dữ liệu và cách biểu diễn có ảnh hưởng trực tiếp đến độ tin cậy của đầu ra.

Với dungeon Zelda, chất lượng không thể chỉ đánh giá bằng hình thức bản đồ. Một hệ tạo sinh phù hợp phải đồng thời bảo đảm logic tiến trình nhiệm vụ, khả năng hoàn thành theo luật chơi, và chất lượng bố cục ở mức tile [2, 3, 4]. Vì vậy, mục tiêu không chỉ là ”sinh được nhiều mẫu” mà là ”sinh được mẫu hợp lệ và có giá trị lối chơi”.

2.1.2 Biểu diễn rời rạc và sinh trong không gian tiềm ẩn

Với dữ liệu phòng ở dạng lưới tile và từ vựng hữu hạn, biểu diễn rời rạc bằng VQ-VAE là lựa chọn phù hợp để học mã ngữ nghĩa [10]. Về nền tảng, cách xây dựng latent và tối ưu hóa mục tiêu tái tạo-khái quát hóa kế thừa trực tiếp từ khung học biểu diễn trong các giáo trình học máy hiện đại [17, 18]. Cách làm này nén thông tin cấu trúc vào không gian tiềm ẩn gọn hơn, từ đó giảm độ khó cho mô-đun sinh phía sau và tăng tính ổn định khi dữ liệu huấn luyện không lớn.

Trên biểu diễn latent rời rạc, diffusion đóng vai trò bộ sinh chính để tạo mẫu có chất lượng cục bộ tốt và đa dạng hình thái [11, 12, 13]. So với sinh trực tiếp trên lưới tile gốc, cách tiếp cận này thường thuận lợi hơn khi cần giữ latent gọn và gắn điều kiện ngữ nghĩa.

Bên cạnh diffusion trong không gian liên tục, hướng diffusion rời rạc cũng đáng chú ý vì làm việc trực tiếp trên các mã rời rạc và trạng thái phân loại. D3PM cho thấy lựa chọn ma trận chuyển trạng thái và cơ chế nhiễu rời rạc có ảnh hưởng rõ đến chất lượng sinh mẫu [19]. Trên các bài toán bố cục có ràng buộc, LayoutDM tiếp tục chỉ ra rằng diffusion rời rạc kết hợp cơ chế tiêm điều kiện ở pha suy luận có thể nâng đáng kể khả năng kiểm soát cấu trúc đầu ra [7]. Ở cùng trục rời rạc, MaskGIT cho thấy chiến lược dự đoán mã bị che theo nhiều vòng lặp có thể rút ngắn đáng kể thời gian giải mã so với cơ chế tự hồi quy tuần tự, đồng thời vẫn giữ chất lượng cấu trúc cạnh tranh ở miền hình ảnh [20]. Kết quả này có ý nghĩa tham chiếu trực tiếp cho các nhánh sinh có che trong bài toán sinh phòng.

2.1.3 Điều kiện hóa theo đồ thị nhiệm vụ

Trong bài toán dungeon, ngữ cảnh phải được xét đồng thời ở hai tầng: cục bộ (liên kết biên, quan hệ phòng kè) và toàn cục (vai trò của phòng trong đồ thị nhiệm vụ). Nếu chỉ dùng thông tin cục bộ, mô hình có thể tạo ra phòng ”đẹp” nhưng lệch với tiến trình nhiệm vụ tổng thể.

Vì vậy, cơ chế điều kiện hóa đa nguồn cùng cơ chế attention chuyển thông tin từ đồ thị sang lưới là lựa chọn được nhiều công trình liên quan cùng cố [21, 22, 23]. Hướng này có thể giúp giảm tình trạng đúng hình thức nhưng sai chức năng, đồng thời tăng tính nhất quán lối chơi giữa các phòng trong cùng một dungeon.

2.1.4 Khung khái niệm và chuẩn hóa thuật ngữ

Bên cạnh nhóm bài báo gốc, Chương 2 còn tham chiếu các giáo trình và tài liệu nền tảng để chuẩn hóa khung khái niệm, thuật ngữ và cách diễn giải. Ở trục PCG, giáo trình của Shaker, Togelius và Nelson cung cấp hệ quy chiếu nhất quán cho các chiến lược dựa trên luật, dựa trên tìm kiếm và dựa trên dữ liệu khi đánh giá hệ tạo sinh [16]. Ở trục học máy, các giáo trình kinh điển về học biểu diễn và mô hình xác suất giúp cố định cách hiểu về không gian tiềm ẩn, điều chuẩn và đánh đổi giữa khớp dữ liệu với khả năng khái quát hóa [17, 18]. Việc kết hợp hai lớp nguồn này giúp phần cơ sở lý thuyết vừa bám sát các công trình hiện đại, vừa giữ được nền tảng phương pháp luận ổn định cho các chương sau.

2.1.5 Tiêu chí phân tích và đánh giá

Đối với hệ lai nơ-ron – ký hiệu, phân lý thuyết không dừng ở việc mô tả mô hình sinh mà còn cần chỉ ra cách đọc kết quả. Các công trình liên quan thường xem xét đồng thời tính hợp lệ cấu trúc, khả năng chơi được, độ đa dạng, độ ổn định khi thay đổi ngân sách và mức độ bền vững trước nhiễu hoặc dịch chuyển miền dữ liệu [24, 25, 3]. Với dungeon Zelda, các tiêu chí này đặc biệt quan trọng vì một mẫu có thể nhìn hợp lý ở mức cục bộ

nhưng vẫn thất bại ở mức tiến trình nhiệm vụ hoặc tạo ra các nghẽn đường đi khó sửa [4, 9]. Cách nhìn theo nhiều trục này là nền tảng để luận văn lựa chọn các phép so sánh phù hợp ở chương thực nghiệm, thay vì chỉ dựa vào một chỉ số đơn lẻ.

2.2 Tổng quan công trình liên quan

Phần này hệ thống các hướng nghiên cứu gần với bài toán của luận văn theo bốn trục chính: đồ thị nhiệm vụ, PCGML cấp phòng, tăng tốc suy luận/sinh rời rạc theo mã, và hệ lai nơ-ron – ký hiệu. Qua đó, luận văn chỉ ra điểm mạnh, giới hạn và mức độ phù hợp của từng hướng đối với dungeon Zelda.

2.2.1 Hướng đồ thị nhiệm vụ và PCG dựa trên tìm kiếm

Các nghiên cứu theo hướng đồ thị nhiệm vụ, mission-space và graph grammar cho thấy ưu điểm rõ rệt của việc mô hình hóa tiến trình nhiệm vụ trước, rồi mới hiện thực hóa hình học phòng [2]. Cách tổ chức này giúp ràng buộc lối chơi được đưa vào sớm, thay vì xử lý hậu kỳ khi lỗi đã xuất hiện trên toàn bản đồ.

Ở nhánh sinh quest riêng, các công trình từ phân tích cấu trúc quest trong MMORPG, sinh quest bằng planning, grammar, mixed-initiative và NLP gần đây cho thấy nhiệm vụ có thể được tạo từ biểu diễn cấu trúc, quy tắc hoặc ngôn ngữ tự nhiên [26, 27, 28, 29, 30]. Tuy nhiên, các hệ này chủ yếu dừng ở mức chuỗi nhiệm vụ hoặc tương tác NPC, chưa giải quyết đồng thời hình học dungeon, đồ thị nhiệm vụ và sửa lỗi ký hiệu trong một pipeline thống nhất.

Trong cùng mạch nghiên cứu, các phương pháp PCG dựa trên tìm kiếm, đặc biệt là nhóm chất lượng-đa dạng như MAP-Elites và CVT-MAP-Elites, cho phép tối ưu đồng thời chất lượng và độ đa dạng [31, 24, 25, 32]. Điều này đặc biệt phù hợp với bài toán sinh dungeon, nơi mục tiêu không chỉ là tìm một nghiệm tốt nhất mà là xây dựng một tập nghiệm phong phú nhưng vẫn chơi được. Ở miền Zelda cụ thể, một nghiên cứu thực nghiệm cho thấy nên đánh giá theo tiến trình khóa-cổng và các chỉ số tuyến tính/độ dư

đường đi thay vì chỉ kiểm tra khả năng giải nhị phân [3].

2.2.2 Thuật toán tìm kiếm để kiểm tra và giải dungeon

Bên cạnh tìm kiếm dùng để sinh nội dung, trong bài toán dungeon còn có một nhánh tìm kiếm khác phục vụ kiểm tra và giải trạng thái chơi. Ở nhánh này, thuật toán không tối ưu hình thái phòng mà duyệt không gian trạng thái của người chơi, vật phẩm và ràng buộc khóa – chìa để kiểm tra xem từ trạng thái khởi đầu có thể đi tới mục tiêu hay không, đồng thời phát hiện các cấu hình dẫn tới bế tắc hoặc thứ tự mở khóa không hợp lệ. Về bản chất, đây là bài toán đi tới đích và thỏa ràng buộc hơn là sinh nội dung thuần túy.

Trong thực hành PCG, các cơ chế kiểm tra này thường xuất hiện dưới dạng duyệt trạng thái, kiểm tra đường đi khả dụng hoặc giải ràng buộc cục bộ để loại bỏ cấu hình mâu thuẫn. WFC là một ví dụ tiêu biểu khi được diễn giải như một bài toán thỏa ràng buộc trong miền tile, còn các quy trình lai nơ-ron – ký hiệu thường dùng một lớp tìm kiếm/xác minh ở hậu kiểm để xác nhận hoặc sửa lại nghiệm do mô hình sinh tạo ra [8, 9]. Vì vậy, nếu phân biệt theo vai trò, tìm kiếm cho sinh dungeon thuộc lớp tối ưu hóa không gian nội dung; còn tìm kiếm cho giải dungeon thuộc lớp kiểm tra tính khả giải, hậu kiểm và sửa lỗi.

2.2.3 Hướng sinh đồ thị học sâu và diffusion rời rạc

Ngoài các tiếp cận grammar và tiến hóa tìm kiếm, hướng sinh đồ thị học sâu gần đây tập trung vào mô hình hóa trực tiếp phân bố trên đồ thị có nhãn nút/cạnh. Graphormer cho thấy các thiên kiến cấu trúc như khoảng cách đường đi ngắn nhất, độ trung tâm và thiên kiến theo cạnh có tác động rõ rệt đến chất lượng biểu diễn đồ thị [33]. Trên nhánh tạo sinh, DiGress mở rộng diffusion sang không gian đồ thị rời rạc và được xem như một mốc so sánh quan trọng cho bài toán sinh đồ thị có thuộc tính phân loại [34].

Ở mức phương pháp tổng quát hơn, D3PM cung cấp nền tảng lý thuyết quan trọng cho diffusion trên không gian rời rạc và giúp lý giải vì sao các thiết kế chuyển trạng thái

khác nhau có thể tạo ra chất lượng sinh mẫu khác nhau ngay cả khi cùng khung huấn luyện [19]. Điều này có giá trị tham chiếu trực tiếp khi đánh giá các biến thể sinh cấu trúc đồ thị nhiệm vụ bằng mô hình học.

Đối với luận văn này, nhóm công trình trên có vai trò chuẩn đổi sánh phương pháp: chúng giúp định vị rõ điểm mạnh/yếu giữa “sinh cấu trúc đồ thị nhiệm vụ bằng grammar kết hợp tìm kiếm” và “sinh cấu trúc đồ thị nhiệm vụ bằng mô hình học đồ thị” trong cùng một ngân sách thực nghiệm. Vì vậy, Chương 4 sẽ sử dụng chúng như mốc tham chiếu khi thảo luận mức cạnh tranh của Khối I theo tiêu chí cùng ngân sách tính toán.

2.2.4 Hướng PCGML ở cấp độ phòng

Ở cấp phòng, các công trình gần đây từ PCGRL đến diffusion cho dữ liệu hạn chế đã báo cáo kết quả hứa hẹn ở bài toán sinh cục bộ [5, 6]. Tổng quan PCGML cũng chỉ ra các vấn đề kéo dài như khan hiếm dữ liệu, khó đồng bộ nhiều lớp ngữ nghĩa và khó chuyển giao phong cách giữa miền dữ liệu khác nhau [4]. Dù các mô hình có kiểm soát ràng buộc như HouseDiffusion và LayoutDM cho thấy tiềm năng rõ rệt, chất lượng hình học vẫn chưa đồng nghĩa với tính đúng tiến trình lối chơi [6, 7]. Vì vậy, chất lượng cục bộ chưa đủ để thay thế một lớp điều phối cấu trúc ở mức cao hơn.

2.2.5 Hướng tăng tốc suy luận và sinh rời rạc theo mã

Khi chuyển từ mô hình có chất lượng cao sang mô hình có thể chạy nhanh trong thời gian chạy, nhóm công trình về nhất quán hóa và chưng cất quỹ đạo suy luận trở thành tham chiếu quan trọng. Consistency Models cho thấy có thể học ánh xạ nhiễu-to-dữ liệu với số bước lấy mẫu rất ít mà vẫn giữ chất lượng cạnh tranh trên các bộ chuẩn thị giác [35]. Trên nền latent diffusion, Latent Consistency Models tiếp tục chứng minh hướng lấy mẫu ít bước ở không gian tiềm ẩn có thể đạt đánh đổi tốt giữa tốc độ và độ trung thực [36].

Song song với nhánh nhất quán hóa, các mô hình sinh mã theo cơ chế tinh chỉnh lặp

theo mặt nạ như MaskGIT cho thấy một hướng tăng tốc khác: thay vì giải mã tuần tự toàn bộ chuỗi mã, mô hình cập nhật song song các vị trí chưa chắc chắn qua nhiều vòng lặp [20]. Với luận văn này, nhóm công trình trên có vai trò chuẩn tham chiếu cho hai nhánh nhanh/có che ở tầng sinh phòng, đồng thời cung cấp tiêu chí rõ ràng để phân tích đánh đổi chất lượng-tốc độ ở Chương 4.

2.2.6 Hướng lai nơ-ron – ký hiệu

Trong bối cảnh PCG, hướng lai nơ-ron – ký hiệu được quan tâm vì kết hợp được hai năng lực bổ sung nhau: khả năng học mẫu linh hoạt của mô hình nơ-ron và khả năng kiểm soát ràng buộc của thành phần ký hiệu [9, 2]. Với các bài toán có ràng buộc lỗi chơi mạnh như Zelda, hướng này đặc biệt phù hợp.

Trong thực hành, kiểm tra hậu kỳ và sửa lỗi ký hiệu không chỉ là bước xử lý tình huống mà còn là một cơ chế quan trọng để nâng độ tin cậy của toàn quy trình; đặc biệt khi kết hợp với các bộ ràng buộc và thủ tục sửa cục bộ như WFC [8, 9].

2.3 Khoảng trống nghiên cứu

Tổng hợp các công trình đã khảo sát cho thấy bốn vấn đề lặp lại khá nhất quán trong bài toán sinh dungeon và quest. Thứ nhất, nhiều phương pháp vẫn nghiêng về một trong hai cực: либо kiểm soát logic toàn cục khá tốt nhưng làm nghèo đa dạng hình thái, либо sinh được bố cục cục bộ phong phú nhưng chưa bảo toàn đầy đủ tính nhất quán của toàn dungeon [1, 4, 24, 25]. Thứ hai, các mô hình ở cấp phòng hoặc cấp đoạn thường không tự động bảo toàn quan hệ phụ thuộc giữa nhiều phòng trong cùng một mission graph, nên chất lượng cục bộ chưa đủ để suy ra tính đúng đắn toàn cục [2, 9, 8]. Thứ ba, so sánh giữa các hệ lai còn thiếu nhất quán do khác nhau về ngân sách tính toán, phạm vi dữ liệu và giao thức đánh giá, khiến kết luận về ưu thế tương đối giữa các phương pháp khó khái quát [4, 24, 25, 32]. Thứ tư, nhiều pipeline chưa mô tả thật rõ vai trò của từng lớp từ đồ thị nhiệm vụ, mô hình sinh phòng đến lớp hậu kiểm tra, làm cho việc tái lập và

diễn giải kết quả bị hạn chế [9, 8].

Từ các khoảng trống đó, luận văn rút ra nhu cầu cho một kiến trúc phân tầng và có thể kiểm chứng, trong đó quest graph giữ vai trò điều phối cấu trúc toàn cục, mô hình sinh trong không gian latent xử lý chi tiết cục bộ, và lớp sửa lỗi ký hiệu bảo toàn khả năng chơi được trước khi tối ưu thêm cho độ đa dạng hình thái. Cách tiếp cận này phù hợp với tinh thần của search-based PCG, PCGML và quality-diversity: ưu tiên nhất quán của gameplay trước, rồi mới mở rộng sang đa dạng và phong phú hình thái sau [1, 4, 5, 24, 25].

2.4 Kết luận chương

Chương 2 đã trình bày nền tảng lý thuyết, tổng quan các hướng nghiên cứu liên quan và xác định khoảng trống mà luận văn hướng tới giải quyết. Trên cơ sở đó, Chương 3 sẽ trình bày chi tiết kiến trúc đề xuất, luồng dữ liệu giữa các khối chức năng và cách hiện thực trong mã nguồn.

CHƯƠNG 3

Phương pháp đề xuất

Chương này trình bày phương pháp đề xuất theo đúng hiện thực trong mã nguồn, nhằm mô tả nhất quán từ quyết định thiết kế đến cơ chế vận hành của hệ thống. Trọng tâm của chương là đặc tả kiến trúc, các đại lượng được tối ưu, luồng tín hiệu điều kiện giữa các khối và các ràng buộc an toàn giúp duy trì chất lượng đầu ra trong bối cảnh dữ liệu huấn luyện hạn chế.

3.1 Mục tiêu và phạm vi chương

Phương pháp được xây dựng để giải đồng thời hai yêu cầu: bảo toàn tiến trình gameplay ở mức toàn dungeon và duy trì chất lượng hình học ở mức từng phòng. Về triển khai, hệ thống tuân theo quy trình *graph-first*: xác lập đồ thị nhiệm vụ trước, sinh phòng có điều kiện theo đồ thị sau, rồi áp lớp hậu kiểm tra ký hiệu để khôi phục và duy trì ràng buộc gameplay. Cách tách tầng này phù hợp với hướng mission-space và nhóm phương pháp lai neural-symbolic trong PCG [2, 9].

Phạm vi chương chỉ bao gồm các nhánh thuộc đường chạy chính của huấn luyện và suy luận: sinh cấu trúc đồ thị nhiệm vụ bằng tiên hóa, mã hóa điều kiện hai luồng, khuếch tán trong không gian tiềm ẩn, hướng dẫn logic và sửa lỗi ký hiệu. Các biến thể ngoài đường chạy chính không được đặc tả trong chương này để giữ tính tái lập và khả năng kiểm chứng của phương pháp.

Để mạch trình bày nhất quán, Chương 3 được tổ chức theo bốn lớp: (i) kiến trúc tổng thể và hợp đồng dữ liệu, (ii) thiết kế chi tiết từng khối, (iii) thuật toán vận hành huấn luyện/suy luận, và (iv) cấu hình triển khai – tái lập. Cấu trúc này giúp tách rõ phần lập luận thiết kế và phần hiện thực trong mã nguồn.

Bảng ký hiệu cốt lõi cho Chương 3. Để giảm tải chuyên ngữ cảnh giữa các mục, Bảng 3.1 gom các ký hiệu được dùng lặp lại trong phần phương pháp.

Bảng 3.1: Ký hiệu dùng xuyên suốt Chương 3	
Ký hiệu	Ý nghĩa
$\mathcal{A}$	Đồ thị kiến trúc ở mức hệ thống (khối xử lý và luồng dữ liệu).
G	Đồ thị nhiệm vụ ở mức nội dung dungeon (tiến trình nhiệm vụ, lock-key, nhánh, mục tiêu).
x_i	Lưới tile của phòng i trong không gian rời rạc 16×11 .
z_i	Biểu diễn latent của phòng i trong không gian liên tục do VQ-VAE mã hóa.
c_i	Vector điều kiện từ đồ thị sang phòng của phòng i sau khi hợp nhất nhánh cục bộ và toàn cục.
D	Dungeon đầu ra sau bước ghép phòng và hậu kiểm tra ký hiệu.
(f, v, ϕ)	Bộ chẩn đoán cá thể đồ thị gồm fitness, mức vi phạm ràng buộc và cờ khả thi.
$\mathcal{L}_{\text{diff}}, \mathcal{L}_{\text{logic}}$	Thành phần mất mát chính trong huấn luyện diffusion có điều kiện và hướng dẫn logic.
$\mathbb{I}_{\text{fallback}}$	Biến chỉ thị kích hoạt cơ chế quay về mô hình giáo viên (fallback) trong suy luận phòng.
$U(a)$	Hàm tiện ích dùng trong P-CBS để xếp hạng trạng thái-kế tiếp ở tầng xác nhận hành vi giới hạn nhận thức.
CGR	Tỉ lệ <i>cognitive gap</i> : phần trăm trường hợp oracle cứng giải được nhưng P-CBS thất bại.

3.2 Kiến trúc tổng thể và hợp đồng dữ liệu

3.2.1 Phân biệt đồ thị kiến trúc và đồ thị nhiệm vụ

Để tránh nhập nhằng thuật ngữ, chương này dùng song song hai lớp đồ thị có vai trò khác nhau:

$$\mathcal{A} = (V_{\mathcal{A}}, E_{\mathcal{A}}), \quad G = (V_G, E_G). \quad (3.1)$$

Trong đó $\mathcal{A}$ là *đồ thị kiến trúc* ở mức hệ thống (nút là các khối xử lý, cạnh là luồng dữ liệu/chỉ đạo), còn G là *đồ thị nhiệm vụ* ở mức nội dung dungeon (nút là phòng/trạng thái nhiệm vụ, cạnh là quan hệ tiến trình gameplay) [2]. Khối I có thể sinh mới G bằng tiến hóa; các khối còn lại nhận G để điều kiện hóa sinh phòng, sửa lỗi, và đánh giá chất lượng.

Quy ước thuật ngữ cho Chương 3. Trong luận văn này, thuật ngữ “graph” được dùng theo nghĩa “đồ thị nhiệm vụ” trong mission-space PCG: bao gồm cấu trúc liên thông và các quan hệ tiến trình có ràng buộc (lock–key, item–gate, switch–state, boss gate) [2, 3, 1]. Khi chỉ phân tích hình thái đồ thị (số nút/cạnh, hệ số phân nhánh, chu trình, phân lớp topo), văn bản dùng cụm “sinh cấu trúc đồ thị” (graph generation) như tên rút gọn [37]. Khi phân tích thứ tự mở tiến trình và điều kiện vượt ải, văn bản dùng “ngữ nghĩa tiến trình” (progression semantics) hoặc “ràng buộc tiến trình nhiệm vụ” (mission progression constraints) để chỉ rõ đây là lớp ngữ nghĩa gameplay vượt ra ngoài nghĩa đồ thị hẹp của lý thuyết đồ thị [2, 3]. Vì vậy, cách gọi đầy đủ cho Khối I trong ngữ cảnh phương pháp là “sinh cấu trúc và ngữ nghĩa tiến trình của đồ thị nhiệm vụ”, còn “graph generation” chỉ dùng ở các đoạn mô tả thuần hình thái cấu trúc.

3.2.2 Mô hình đồ thị kiến trúc 7 khối

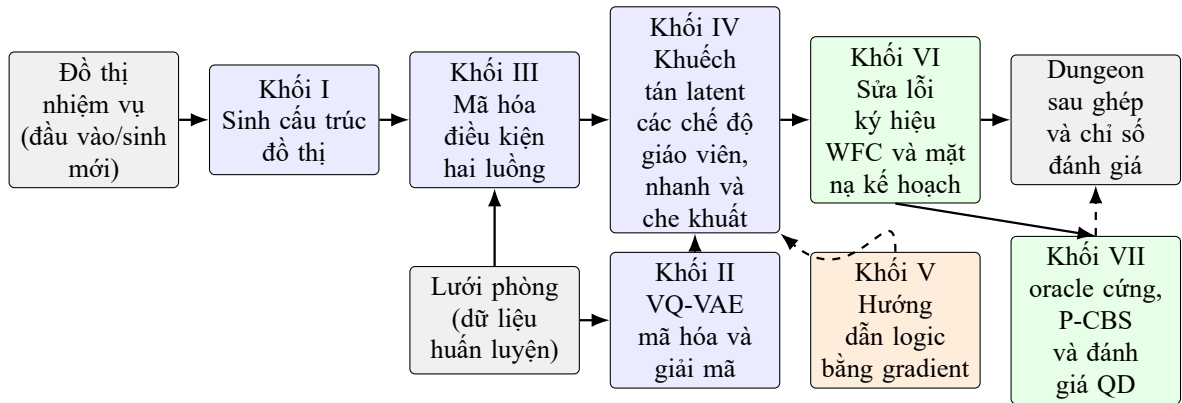
Để đặc tả kiến trúc ở mức cấu trúc, ta mô hình hóa pipeline bằng một đồ thị có hướng:

$$\mathcal{A} = (V_{\mathcal{A}}, E_{\mathcal{A}}), \quad V_{\mathcal{A}} = \{B_1, \dots, B_7\}, \quad (3.2)$$

trong đó B_1 đến B_7 lần lượt tương ứng các khối I đến VII ở Hình 3.1. Tập cạnh được tách thành hai phần:

$$E_{\mathcal{A}} = E_{\text{core}} \cup E_{\text{opt}}, \quad (3.3)$$

với E_{core} là các luồng bắt buộc trên đường chạy chính (cấu trúc đồ thị nhiệm vụ $\rightarrow$ conditioning $\rightarrow$ diffusion $\rightarrow$ repair), còn E_{opt} là các luồng tùy chọn dùng cho hướng dẫn logic hoặc đánh giá quality-diversity.



Hình 3.1: Kiến trúc 7 khối của quy trình sinh nơ-ron-ký hiệu theo hướng graph-first. Mũi tên nét đứt biểu diễn các nhánh hướng dẫn logic và đánh giá tùy cấu hình.

Trong phân rã chức năng, Khối I chịu trách nhiệm tiên nghiệm cấu trúc đồ thị của G ; Khối II chuẩn hóa không gian latent; Khối III hợp nhất điều kiện cục bộ-toàn cục theo đồ thị; Khối IV thực hiện lấy mẫu khuếch tán; Khối V bổ sung ràng buộc logic khả vi; Khối VI phục hồi ràng buộc ký hiệu ở tầng tile; và Khối VII thực hiện xác nhận hậu sinh theo ba vai trò bổ sung: oracle cơ học cứng, xác nhận hành vi giới hạn nhận thức P-CBS và đánh giá chất lượng-đa dạng ở mức dungeon. Cách tách khối này tạo ra giao diện rõ ràng cho phân tích loại trừ theo thành phần ở Chương 4.

3.2.3 Ma trận hợp đồng dữ liệu theo khối

Để bảo đảm truy vết được trách nhiệm của từng khối trong đường chạy chính, Bảng 3.2 tóm tắt hợp đồng vào-ra, cơ chế chặn lỗi và chỉ số giám sát tương ứng.

Bảng 3.2: Ma trận hợp đồng dữ liệu và cơ chế an toàn theo từng khối

Khối	Hợp đồng vào-ra cốt lõi	Cơ chế chặn lỗi/chuyển trạng thái	Mục tiêu hoặc chỉ số chính
B_1	Vào: luật, seed, hồ sơ ràng buộc. Ra: đồ thị nhiệm vụ khả thi G và bộ mô tả cấu trúc/tiến trình.	Chọn nghiệm khả thi-ưu tiên; hậu kiểm tra liên thông, start-goal, virtual node.	Fitness đa mục tiêu, mức vi phạm v , cờ khả thi ϕ .
B_2	Vào: lưới phòng rời rạc. Ra: latent z và giải mã tile.	Kiểm tra shape chuẩn, bỏ batch non-finite.	$\mathcal{L}_{\text{recon}}$, $\mathcal{L}_{\text{VQ}}$, tỷ lệ tile hiếm.
B_3	Vào: $(X, E_{idx}, E_f, \text{TPE})$ cùng latent lân cận. Ra: điều kiện phòng c_i .	Kiểm tra lược đồ tensor và dừng sớm khi sai kích thước.	Độ ổn định điều kiện hóa theo lớp phòng, lỗi lược đồ bằng 0.
B_4	Vào: z_t , timestep t , điều kiện c . Ra: latent mẫu và dự đoán nhiễu/vận tốc.	Kẹp CFG theo miền chung cất, trọng số Min-SNR, cắt gradient.	$\mathcal{L}_{\text{diff}}$, tốc độ mẫu sinh, tỷ lệ quay về mô hình giáo viên.
B_5	Vào: latent trung gian và plan trace. Ra: gradient logic và $\mathcal{L}_{\text{logic}}$.	Warmup theo epoch; tắt logic khi thành phần không khả dụng.	Grid reach, trace consistency, anchor consistency.

Khối	Hợp đồng vào-ra cốt lõi	Cơ chế chặn lỗi/chuyển trạng thái	Mục tiêu hoặc chỉ số chính
B_6	Vào: lưới nơ-ron, mặt nạ kế hoạch, marker đồ thị. Ra: lưới phòng đã sửa và hợp lệ biên-cửa.	Ép biên, sửa có dẫn dắt bởi WFC, giới hạn số vòng sửa.	Repair rate, số lỗi cửa/đảo, hợp lệ kích thước phòng.
B_7	Vào: dungeon đã ghép và hợp đồng validation đã chuẩn hóa. Ra: phán quyết oracle, chỉ số P-CBS, cập nhật kho QD và thống kê hậu sinh.	Chuẩn hóa lưới ghép, loại bỏ token ngoài bảng từ điển/VOID nội suy, ép đúng một start-goal, chỉ thêm vào kho khi oracle cứng giải được.	Hard-solvable rate, cognitive-gap rate, navigation entropy/confusion, coverage, QD-score.

3.2.4 Dòng chảy đầu-cuối của quy trình

Gọi G là đồ thị nhiệm vụ, N là số phòng vật lý sau bước lọc nút ảo và D là dungeon đầu ra. Hệ thống thực hiện ánh xạ:

$$D = \mathcal{S}\left(\{\mathcal{R}(\mathcal{N}(C_i, G_i))\}_{i=1}^N\right), \quad (3.4)$$

trong đó $\mathcal{N}$ là khối sinh phòng nơ-ron có điều kiện, $\mathcal{R}$ là khối sửa lỗi ký hiệu theo ràng buộc cục bộ, và $\mathcal{S}$ là khối ghép phòng để tạo bản đồ toàn cục.

Trong thực thi, quy trình gồm ba lớp xử lý nối tiếp. Ở lớp đồ thị, hệ thống hoặc nhận đồ thị nhiệm vụ từ đầu vào hoặc tự sinh cấu trúc đồ thị nhiệm vụ bằng tìm kiếm tiến hóa, rồi chuẩn hóa thứ tự nút để cố định ánh xạ giữa định danh phòng và chỉ số nút. Ở lớp phòng, các phòng được sinh theo lớp topo của đồ thị có hướng [37], sau đó nhóm

theo dạng latent để gom lô mà không phá phụ thuộc tiền nhiệm. Ở lớp hậu xử lý, đầu ra nơ-ron được kiểm tra và sửa cục bộ theo mặt nạ kế hoạch trước khi ghép dungeon, nhờ đó tính nhất quán tiến trình được bảo toàn khi chuyển từ mức phòng sang mức toàn bản đồ.

3.2.5 Biểu diễn dữ liệu và schema đầu vào

Đồ thị nhiệm vụ được biểu diễn dưới hai dạng: vô hướng để duy trì liên thông toàn cục và có hướng để mô tả phụ thuộc tiến trình. Từ đồ thị này, hệ thống trích xuất tensor đặc trưng nút 14 chiều, đặc trưng cạnh 16 chiều và mã hóa vị trí topo 8 chiều. Không gian phòng sử dụng lưới rời rạc 16×11 với 44 lớp tile; sau đó Khối II ánh xạ sang latent chuẩn hóa $64 \times 4 \times 3$ thông qua VQ-VAE [10]. Thiết kế ở mức latent giúp diffusion học phân bố ổn định hơn trong thiết lập dữ liệu phòng quy mô nhỏ, đồng thời vẫn duy trì khả năng can thiệp ký hiệu ở tầng tile [13]. Chi tiết cơ chế hợp nhất điều kiện cục bộ-toàn cục của Khối III được trình bày riêng trong Mục 3.3 để tách bạch lớp hợp đồng dữ liệu và lớp thiết kế thuật toán của từng khối.

3.2.6 Tiền xử lý dữ liệu và chính sách chia tập

Về tiền xử lý, quy trình chuẩn hóa dữ liệu về cùng một hợp đồng hình học-ngữ nghĩa trước khi huấn luyện: phòng được đưa về kích thước chuẩn 16×11 , tile id được ánh xạ vào từ vựng 44 lớp, và các kênh điều kiện theo biên/hàng xóm được dựng đồng nhất với lược đồ ở Mục 3.2.5. Cùng lúc, dữ liệu đồ thị được chuẩn hóa thứ tự nút để tránh lệch chỉ số giữa pha huấn luyện và pha suy luận.

Để hạn chế rò rỉ thông tin giữa các tập, giao thức chia tập tuân thủ tính rời nhau theo nguồn dữ liệu:

$$\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{val}} = \emptyset, \quad \mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset, \quad \mathcal{D}_{\text{val}} \cap \mathcal{D}_{\text{test}} = \emptyset. \quad (3.5)$$

Ở mức phòng, các mẫu cùng xuất xứ đồ thị nhiệm vụ được ràng buộc vào cùng split:

$$\forall r_a, r_b : \text{src}(r_a) = \text{src}(r_b) \Rightarrow \text{split}(r_a) = \text{split}(r_b). \quad (3.6)$$

Nguyên tắc này giúp tránh trường hợp mô hình thấy một phần của cùng mission graph ở train và phần còn lại ở test, từ đó làm sai lệch kết luận về khả năng khái quát.

3.2.7 Ràng buộc triển khai và bất biến dữ liệu

Trong hiện thực hệ thống, chất lượng đầu ra không chỉ được kiểm soát bởi hàm mục tiêu mà còn bởi tập bất biến dữ liệu được kiểm tra xuyên suốt ở các điểm vào/ra của quy trình. Ở mức hình học phòng, trạng thái hợp lệ được duy trì bởi:

$$x_i \in \{0, \dots, 43\}^{16 \times 11}, \quad z_i \in \mathbb{R}^{64 \times 4 \times 3}. \quad (3.7)$$

Ở mức ràng buộc biên, vector điều kiện phòng được cố định 8 chiều theo thủ tục dựng ràng buộc biên:

$$b_i = [h_N, r_N, h_S, r_S, h_E, r_E, h_W, r_W] \in \{0, 1\}^8, \quad r_d \leq h_d \ (d \in \{N, S, E, W\}). \quad (3.8)$$

Gọi $\delta_d(\hat{x}_i) \in \{0, 1\}$ là biến chỉ thị “có cửa ở cạnh d ” trên phòng sinh ra $\hat{x}_i$, khi đó hợp đồng ràng buộc theo cạnh được viết:

$$r_d \leq \delta_d(\hat{x}_i) \leq h_d, \quad d \in \{N, S, E, W\}. \quad (3.9)$$

Biểu thức trên phản ánh đúng hành vi triển khai: cạnh không có hàng xóm bị ép thành tường biên; cạnh có cửa bắt buộc được khôi phục thông qua lớp sửa biên và lớp sửa ký hiệu trước khi trả kết quả phòng.

Ở mức graph-to-room conditioning, các tensor đầu vào được duy trì đúng schema:

$$X \in \mathbb{R}^{N \times 14}, \ E_{\text{idx}} \in \mathbb{N}^{2 \times M}, \ E_f \in \mathbb{R}^{M \times 16}, \ \text{TPE} \in \mathbb{R}^{N \times 8}, \quad (3.10)$$

và khi batch hóa cho diffusion phải thỏa:

$$\mathbf{B}_{\text{cond}} \in \mathbb{R}^{B \times 8}, \quad \mathbf{T}_{\text{topo}} \in \mathbb{R}^{B \times C \times H \times W}. \quad (3.11)$$

Các vi phạm shape tại các điểm này được phát hiện sớm bằng kiểm tra kiểu-kích thước và cơ chế báo lỗi sớm, thay vì để lỗi lan đến bước lấy mẫu; nhờ đó giảm đáng kể trường hợp “sinh được nhưng sai hợp đồng dữ liệu”.

3.3 Thiết kế chi tiết các khối trong pipeline

3.3.1 Nguyên tắc lựa chọn thiết kế giữa các phương án thay thế

Để làm rõ phần “vì sao chọn thiết kế này”, luận văn dùng năm nguyên tắc xuyên suốt khi quyết định kiến trúc:

- (i) **Tách đồ thị nhiệm vụ và hình học phòng:** ưu tiên kiểm soát tiến trình gameplay ở cấp đồ thị trước khi tối ưu chi tiết hình học ở cấp phòng.
- (ii) **Rời rạc hóa latent trước khi khuếch tán:** chọn VQ-VAE để nén ngữ nghĩa tile vào codebook ổn định, giảm độ khó cho khối diffusion trong thiết lập dữ liệu nhỏ.
- (iii) **Điều kiện hóa hai luồng cục bộ-toàn cục:** giữ đồng thời tín hiệu lân cận trực tiếp và vai trò phòng ở cấp dungeon để giảm xung đột biên-cửa.
- (iv) **Ổn định hóa huấn luyện hơn là tối ưu ngắn hạn:** dùng Min-SNR, warmup logic và kẹp CFG theo miền distillation để tránh nhiễu gradient ở giai đoạn đầu.
- (v) **An toàn suy luận theo lớp phòng thủ:** cho phép sampler nhanh hoạt động, nhưng luôn đặt hậu kiểm tra ký hiệu và cơ chế quay về mô hình giáo viên làm tầng bảo hiểm cuối.

Năm nguyên tắc này cũng là cơ sở để thiết kế ablation ở Chương 4: mỗi nguyên tắc tương ứng một hoặc nhiều cờ bật/tắt độc lập, nhờ đó có thể ước lượng đóng góp biên của từng lớp quyết định kiến trúc.

3.3.2 Mô hình sinh đồ thị nhiệm vụ (Khối I): cấu trúc và tiến trình

Khối I sử dụng bộ sinh đồ thị nhiệm vụ tiến hóa dựa trên grammar, trong đó genotype là chuỗi luật và phenotype là đồ thị nhiệm vụ. Khi mô tả hình thái của đồ thị, chẳng hạn số nút, số cạnh, mức phân nhánh và số chu trình, luận văn dùng nhóm chỉ số sinh cấu trúc đồ thị; khi mô tả các quan hệ khóa-cổng-vật phẩm-thứ tự vượt ải, luận văn dùng nhóm chỉ số ngữ nghĩa tiến trình hoặc ràng buộc tiến trình nhiệm vụ [2, 3, 37]. Trong đối sánh với hướng học thuần đồ thị, DiGress là baseline quan trọng cho sinh đồ thị rời rạc có nhãn nút/cạnh [34]. Hàm thích nghi kết hợp độ khớp với đường cong mục tiêu (tension/difficulty), tính khả thi của ràng buộc và các đặc trưng cấu trúc, nhờ đó tránh việc quần thể sớm hội tụ về một kiểu đồ thị duy nhất. Trong cấu hình vận hành, không gian luật được mở rộng cùng cơ chế phối trộn chuyển trạng thái để cân bằng giữa giả định thiết kế kiểu Zelda và yêu cầu đa dạng cấu trúc, phù hợp với tinh thần chất lượng-đa dạng của MAP-Elites/CVT [24, 25, 32].

Cơ chế chọn nghiệm và áp lực ràng buộc

Để làm tường minh lớp sinh cấu trúc đồ thị nhiệm vụ, ta mô hình hóa mỗi cá thể bằng bộ ba: chuỗi luật (genome), đồ thị sinh ra (phenotype), và bộ chẩn đoán khả thi. Cụ thể:

$$g = (r_1, \dots, r_L), \quad \mathcal{G} = \text{EXECUTE}(g), \quad (f, v, \phi) = \text{EVALUATEGRAPH}(\mathcal{G}), \quad (3.12)$$

trong đó L là độ dài genome, $f \in [0, 1]$ là fitness, $v \geq 0$ là tổng mức vi phạm ràng buộc, và $\phi = \mathbf{1}[v \leq 10^{-9}]$ là cờ khả thi. Hàm EVALUATEGRAPH kết hợp độ khớp tension curve, descriptor cấu trúc và các hạng vi phạm (độ dài đường chính, dải số nút/cạnh, realism, directionality, rejection ratio, Pareto loops/branching), đúng với quy trình đánh giá đồ thị ở tầng hiện thực.

Trong bước chọn nghiệm, bộ sinh dùng thứ tự khả thi-ưu tiên kiểu Deb [38]:

$$\kappa(i) = (\mathbf{1}[\neg\phi_i], (1 - \phi_i) v_i, -f_i, \varepsilon_i), \quad (3.13)$$

với ε_i là lỗi realism của cấu trúc đồ thị. Cá thể có khóa từ điển nhỏ hơn sẽ được ưu tiên trong tournament và survivor selection, nên quần thể khả thi luôn được xếp trước quần thể không khả thi.

Thuật toán 1 Sinh đồ thị nhiệm vụ bằng GA hoặc CVT-emitter với ưu tiên nghiệm khả thi

Đầu vào: Mục tiêu đường cong τ , cấu hình (P, G, L) , chiến lược $s \in \{GA, CVT-emitter\}$

Đầu ra: Đồ thị nhiệm vụ $\mathcal{G}^*$

```

1: Nếu  $s = CVT-emitter$  thì
2:    $B \leftarrow P \cdot G$  ▷ ngân sách đánh giá
3:   Khởi tạo CVT archive
4:   Đối với mỗi  $e = 1$  đến  $B$  thực hiện
5:     Phát sinh một genome từ emitter (khởi tạo ngẫu nhiên hoặc dẫn hướng từ archive)
6:      $\mathcal{G} \leftarrow EXECUTE(g)$ ;  $(f, v, \phi) \leftarrow EVALUATEGRAPH(\mathcal{G})$ 
7:     Cập nhật archive và cá thể tốt nhất theo khóa  $\kappa$ 
8:   end Đối với mỗi
9: Ngược lại
10:  Khởi tạo quần thể ban đầu kích thước  $P$ 
11:  Đối với mỗi  $t = 1$  đến  $G$  thực hiện
12:    Đánh giá toàn bộ quần thể:  $EXECUTE \rightarrow EVALUATEGRAPH$ 
13:    Nếu  $\max f \geq 0.95$  thì
14:      break ▷ dừng sớm khi đã hội tụ
15:    end Nếu
16:    Sinh thế hệ con bằng tournament selection, lai ghép và đột biến thích nghi
17:    Đánh giá offspring
18:    Chọn cá thể sống sót theo quy tắc khả thi-ưu tiên trong mô hình  $(\mu + \lambda)$ 
19:  end Đối với mỗi
20: end Nếu
21: Hậu xử lý đồ thị tốt nhất: sửa tiến trình, lọc virtual node, kiểm tra hợp lệ cấu trúc đồ thị
22: Trả về:  $\mathcal{G}^*$ 

```

Thuật toán 1 tóm tắt cách Khối I vận hành trong thực tế: mục tiêu chất lượng được tối ưu đồng thời với áp lực ràng buộc xuyên suốt từ lúc áp luật grammar, đánh giá cá thể, chọn sống sót cho đến hậu kiểm đầu ra.

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Gọi $n = |V|$, $m = |E|$, P là kích thước quần thể, G là số thế hệ và L là độ dài genome. Tính đúng của thuật toán được bảo đảm bởi ba lớp kiểm soát ràng buộc: cơ chế chặn các hành động không hợp lệ theo trạng thái hiện tại ở pha tạo phenotype; cờ khả thi $\phi = \mathbf{1}[v \leq 10^{-9}]$ ở pha đánh giá; và

hậu kiểm tra tính hợp lệ của cấu trúc đồ thị (liên thông, start/goal, độ dài đường đi tối thiểu, hợp lệ boss-goal, loại virtual node) trước khi trả kết quả cuối. Quá trình tìm kiếm dừng khi nhánh GA chạy đủ G thế hệ hoặc đạt ngưỡng hội tụ fitness tốt nhất (≥ 0.95), còn nhánh CVT-emitter dừng khi dùng hết ngân sách $P \cdot G$. Từ chi phí thực thi, đánh giá và chọn lọc, độ phức tạp được chặn trên bởi:

$$\mathcal{O}(PG (C_{\text{exec}} + C_{\text{eval}}) + PG \log P) \quad (3.14)$$

cho nhánh GA, và

$$\mathcal{O}(PG (C_{\text{exec}} + C_{\text{eval}})) \quad (3.15)$$

cho nhánh CVT-emitter.

3.3.3 Khối mã hóa điều kiện graph-to-room (Khối III)

Khối III ánh xạ ngữ cảnh đồ thị nhiệm vụ sang vector điều kiện cho từng phòng đích. Với phòng i , bộ điều kiện được xây từ hai nguồn: ngữ cảnh cục bộ và ngữ cảnh toàn cục.

$$c_i^{\text{local}} = f_{\text{local}}([z_N, z_S, z_E, z_W, b_i, p_i]), \quad (3.16)$$

$$H = \text{GNN}(G; X, E, \text{TPE}), \quad c_i^{\text{global}} = H_i, \quad (3.17)$$

$$c_i = \text{Attn}(q = c_i^{\text{local}}, K = H, V = H) + f_{\text{fuse}}([c_i^{\text{local}}, c_i^{\text{global}}]). \quad (3.18)$$

Trong đó $z_{N,S,E,W}$ là latent của các phòng lân cận theo bốn hướng, b_i là ràng buộc biên và p_i là đặc trưng vị trí phòng. Nhánh toàn cục hỗ trợ nhiều biến thể GNN; cấu hình mặc định ưu tiên GPS cho bài toán đồ thị có tín hiệu toàn cục mạnh [23]. Ngoài ra, các thiên kiến cấu trúc theo kiểu Graphormer, như khoảng cách đường đi ngắn nhất và độ lệch cấu trúc theo quan hệ cạnh, cũng là cơ sở lý thuyết để củng cố lớp mã hóa đồ thị trong khối này [33]. Tầng attention đóng vai trò căn chỉnh thông tin toàn cục với ngữ cảnh cục bộ của từng phòng, từ đó giảm mâu thuẫn giữa ràng buộc biên và vai trò gameplay trong toàn bộ dungeon.

3.3.4 Khối sinh nơ-ron: VQ-VAE (Khối II) và khuếch tán tiềm ẩn (Khối IV)

Khối II đóng vai trò cầu nối giữa không gian tile rời rạc và không gian latent liên tục, còn Khối IV học quá trình khử nhiễu có điều kiện trên latent theo khung DDPM/DDIM [11, 12]. Ở pha suy luận, mô hình sử dụng classifier-free guidance (CFG), với hệ số guidance s_t thay đổi theo lịch chạy và được giữ trong miền đã hiệu chỉnh khi dùng bộ lấy mẫu nhanh [39]. Trong hiện thực mã nguồn, nhánh lấy mẫu rút gọn luôn bị giới hạn bởi miền CFG đã được hiệu chỉnh nên không thể đẩy hệ số guidance vượt quá ngưỡng an toàn.

$$\tilde{\epsilon}_\theta(x_t, t, c) = \epsilon_\theta(x_t, t, \emptyset) + s_t(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \emptyset)), \quad (3.19)$$

Ở pha huấn luyện, diffusion loss được tái trọng số theo Min-SNR để giảm lệch trọng số giữa các timestep:

$$\text{SNR}_t = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}, \quad w_t = \frac{\min(\text{SNR}_t, \gamma)}{\text{SNR}_t + \varepsilon}, \quad \mathcal{L}_{\text{diff}} = \mathbb{E}_{t, \epsilon} [w_t \|\hat{y}_t - y_t\|_2^2], \quad (3.20)$$

với y_t là nhiễu mục tiêu (hoặc vận tốc v khi bật v-prediction) và γ là ngưỡng Min-SNR [40].

3.3.5 Khối hướng dẫn logic và sửa lỗi ký hiệu (Khối V–VI)

Khối V bổ sung tín hiệu logic vào huấn luyện theo cơ chế *warmup* nhằm tránh gây nhiễu khi mô hình chưa ổn định:

$$\mathcal{L}_{\text{total}} = \alpha_{\text{visual}} \mathcal{L}_{\text{diff}} + \mathbf{1}[e \geq e_{\text{warmup}}] \alpha_{\text{logic}} \mathcal{L}_{\text{logic}} + \lambda_{\text{stage}} \mathcal{L}_{\text{stage}}. \quad (3.21)$$

Ở nhánh tăng cường cho puzzle nhiều bước, λ_{stage} là trọng số của mất mát ngữ nghĩa stage được bật có điều kiện. Hạng tử này chỉ hoạt động khi batch hiện tại thật sự mang metadata `puzzle_stage_condition` và cấu hình huấn luyện bật supervision tương ứng; nếu không, $\lambda_{\text{stage}} = 0$. Trong triển khai hiện tại, semantic head phụ trợ dự đoán trực tiếp

từ room logits đã giải mã bốn thuộc tính: họ cổng (*gate family*), cờ *sequence-required*, số stage hữu hiệu, và chuỗi stage có thứ tự. Viết gọn:

$$\mathcal{L}_{\text{stage}} = \mathcal{L}_{\text{gate}} + \mathcal{L}_{\text{seq}} + \mathcal{L}_{\text{count}} + \mathcal{L}_{\text{slot}}, \quad (3.22)$$

trong đó $\mathcal{L}_{\text{slot}}$ chỉ được cộng trên các vị trí stage hợp lệ thực sự tồn tại trong chuỗi mục tiêu. Cấu trúc này biến nhánh diffusion từ “được điều kiện hóa bởi stage” sang “bị ràng buộc phải tái tạo ngữ nghĩa stage đúng thứ tự” ở ngay tầng logit.

Trong suy luận theo kiểu DDIM, guidance logic tác động trực tiếp lên dự đoán nhiễu:

$$\hat{\epsilon}'_{\theta} = \hat{\epsilon}_{\theta} + \sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \mathcal{L}_{\text{logic}}, \quad (3.23)$$

rồi mới cập nhật lại $\hat{x}_0$. Cách viết này phản ánh đúng thao tác trong Khối IV khi logic guidance được bật.

Sau giải mã, Khối VI xử lý phòng theo chuỗi chuẩn hóa gồm làm sạch dữ liệu, ép biên và sửa cục bộ có điều kiện theo mặt nạ kế hoạch. Thành phần inpainting có dẫn dắt bởi WFC được dùng như cơ chế sửa cục bộ khi phát hiện vùng vi phạm ràng buộc [8]. Vì mặt nạ kế hoạch được suy ra từ ngữ nghĩa đồ thị (vai trò phòng, cửa bắt buộc, quan hệ vào/ra), bước sửa lỗi không chỉ cải thiện hình học mà còn phục hồi tính đúng của tiến trình gameplay.

Mục tiêu của thuật toán là giữ nhất quán giữa tầng nơ-ron và tầng ký hiệu: đầu vào là lưới nơ-ron cùng ngữ cảnh đồ thị phòng, đầu ra là lưới đã chuẩn hóa ngữ nghĩa, thỏa ràng buộc biên-cửa và bảo toàn khả năng giải cục bộ. Trình tự xử lý được cố định theo chuỗi chuẩn hóa $\rightarrow$ ép biên $\rightarrow$ sửa có phản hồi theo mặt nạ kế hoạch $\rightarrow$ phủ marker đồ thị $\rightarrow$ kiểm tra hợp lệ, nhờ đó mọi thay đổi đều có thể truy vết và kiểm chứng.

Thuật toán 2 là lớp nối giữa chất lượng hình học và tính đúng gameplay. Nếu thiếu lớp này, các sai lệch cục bộ về cửa, biên hoặc đường đi có thể lan sang bước ghép dungeon và làm giảm độ tin cậy của đánh giá toàn cục.

Thuật toán 2 Sửa phòng có phản hồi ký hiệu ở tầng hậu xử lý

Đầu vào: Lưới nơ-ron đầu vào x_i^{neural} , đồ thị phòng G_i , cặp start-goal (s_i, g_i)

Đầu ra: Grid cuối x_i^{final} thỏa ràng buộc biên và đường đi

- 1: $x_i^{\text{final}} \leftarrow \text{SANITIZESEMANTICIDS}(x_i^{\text{neural}})$
 - 2: $x_i^{\text{final}} \leftarrow \text{STRIPVOLATILESEMANTICS}(x_i^{\text{final}}, G_i)$
 - 3: $x_i^{\text{final}} \leftarrow \text{ENFORCEBOUNDARYSHELL}(x_i^{\text{final}}, G_i)$
 - 4: $m_i \leftarrow \text{BUILDROOMPLANTRACE}(G_i, s_i, g_i)$
 - 5: **Nếu** cơ chế sửa hậu kỳ được kích hoạt **thì**
 - 6: $x_i^{\text{cand}} \leftarrow \text{REPAIRROOMWITHFEEDBACK}(x_i^{\text{final}}, m_i)$
 - 7: **Nếu** bước sửa tạo ra cấu hình hợp lệ **thì**
 - 8: $x_i^{\text{final}} \leftarrow x_i^{\text{cand}}$
 - 9: **end Nếu**
 - 10: **end Nếu**
 - 11: $x_i^{\text{final}} \leftarrow \text{OVERLAYGRAPHMARKERS}(x_i^{\text{final}}, G_i)$
 - 12: $\text{VALIDATEROOMDIMENSIONS}(x_i^{\text{final}})$
 - 13: **Trả về:** x_i^{final}
-

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Ở tầng sửa hậu kỳ, tính đúng của đầu ra phòng được bảo đảm khi đồng thời giữ đúng kích thước phòng, thỏa hợp đồng biên-cửa theo ngữ cảnh đồ thị, và không làm suy giảm khả năng cục bộ sau bước repair cùng overlay marker. Thuật toán dừng khi hoàn tất chuỗi thao tác chuẩn hóa hữu hạn; riêng nhánh repair bị chặn trên bởi số vòng thử theo cấu hình runtime. Do đó, độ phức tạp theo phòng là:

$$\mathcal{O}(HW + R C_{\text{rep}}(H, W)), \quad (3.24)$$

Trong phiên bản cuối của P-CBS, hai số hạng ξ và ζ mô hình hóa *progression-affordance memory*: tác tử có thể nhớ lại một cửa khóa, bombable gate hay puzzle anchor đã quan sát trước đó sau khi inventory thay đổi, đồng thời vẫn bị phạt nếu affordance đó không còn được hỗ trợ tốt bởi bộ nhớ làm việc. với R là số vòng repair thực thi.

Ngữ pháp puzzle trạng thái nhiều bước ở tầng hậu xử lý. Phiên bản hiện tại của Khối VI không còn xem “puzzle room” như một phòng có vài vật cản ngẫu nhiên quanh marker ngữ nghĩa. Thay vào đó, tầng hậu xử lý duy trì một *puzzle plan* hữu hạn cho từng phòng stateful, trong đó mỗi phòng được gán một chuỗi stage có thứ tự như collect_key,

`collect_item`, `step_on_puzzle` hoặc `push_block_to_switch`. Các stage này được sinh từ ngữ nghĩa cạnh và vai trò phòng trong đồ thị nhiệm vụ, sau đó được chuẩn hóa thành metadata dùng chung cho cả bộ sinh lẫn validator. Vì vậy, một cửa `DOOR_PUZZLE` không còn chỉ phụ thuộc vào hình dạng cục bộ của phòng, mà phụ thuộc vào việc chuỗi stage tương ứng đã hoàn tất trong trạng thái chơi hay chưa. Thiết kế này đưa hệ thống từ “trang trí puzzle” sang “puzzle có tiến trình trạng thái” theo hướng hybrid: ngữ pháp và validator chịu trách nhiệm tính đúng, còn mô hình nơ-ron chịu trách nhiệm hình học và bề mặt thị giác của phòng.

Trong phiên bản tăng cường gần nhất, cùng metadata stage này còn được dùng ở pha huấn luyện như một tín hiệu giám sát tường minh. Cụ thể, nhánh nghiên cứu `stageconditioned_semantics_v2` không chỉ nhận stage token và stage trace như đầu vào điều kiện hóa, mà còn dùng semantic head phụ trợ để buộc room logits sau giải mã dự đoán đúng *gate family*, *sequence-required*, *stage count* và thứ tự các *stage slots*. Điều này làm cho tuyên bố “learned multi-step puzzle semantics” có cơ sở mô hình học rõ ràng hơn so với phiên bản chỉ có hybrid grammar ở runtime. Tuy nhiên, vì các checkpoint cũ không có hạng mất mát này, chúng không được dùng như bằng chứng cho tuyên bố trên.

3.3.6 Khối xác nhận hậu sinh: oracle cứng, P-CBS và giao thức bàn giao validation (Khối VII)

Khối VII là điểm chốt của pipeline sau khi các phòng đã được ghép thành dungeon hoàn chỉnh. Ở đây luận văn tách rõ hai lớp xác nhận có vai trò khác nhau thay vì dùng một bộ giải duy nhất cho mọi mục tiêu: (i) *oracle cứng* để quyết định tính đúng cơ học của dungeon, và (ii) *Persona-Driven Cognitive Bounded Search (P-CBS)* để đo mức ma sát nhận thức của các persona người chơi. Cách tách này phù hợp với thực hành trong automated playtesting: các procedural persona cổ điển chủ yếu dùng MCTS hoặc RL để mô phỏng lối chơi, trong khi các bộ giải trạng thái tối ưu được dùng để kiểm tra tính beatable/solvable về mặt cơ học [41, 42]. Trong phiên bản hiện tại của hệ thống, oracle

cứng được báo cáo theo *hybrid mechanical contract: graph-guided room oracle + graph progression validator + kiểm tra soft-lock quyết định*. Phép kiểm A* nguyên khối trên lưới ghép vẫn được giữ lại, nhưng với vai trò *stress probe* nghiêm ngặt hơn vì trên lát cắt puzzle nhiều trạng thái hiện tại nó thường chạm ngưỡng timeout trước khi phủ hết không gian trạng thái.

Hợp đồng bàn giao lưới ghép trước validation. Để tránh việc mô-đun validator nhận đầu vào khác chuẩn so với mô tả học thuật, lưới ghép phải đi qua một bước chuẩn hóa tường minh tương ứng với thủ tục `PrepareValidationGrid` trong mã nguồn. Bước này thực hiện bốn thao tác hữu hạn: ép kiểu về ma trận số nguyên hai chiều, ánh xạ mọi token ngoài bảng từ điển semantic về tile hợp lệ, lấp các vùng VOID kín không nối với biên ngoài, và chuẩn hóa đúng một cặp START/TRIFORCE. Viết gọn:

$$D^* = \Pi_{\text{valid}}(D), \quad (3.25)$$

trong đó Π_{valid} là phép chiếu từ lưới ghép thô sang miền hợp lệ của bộ từ vựng tile và ràng buộc terminal. Cơ chế này giải quyết trực tiếp lớp lỗi *VOID leak* hoặc *unmapped semantic token* vốn có thể làm sai lệch phán quyết validator nếu mỗi script tự xử lý ad hoc.

Oracle cứng cho tính đúng cơ học. Sau chuẩn hóa, oracle cứng chỉ công nhận dungeon là hợp lệ khi đồng thời thỏa ba điều kiện:

$$\phi_{\text{hard}}(D^*) = \mathbf{1}[\phi_{\text{guided}}(G, D^*) \wedge \phi_{\text{prog}}(G, D^*) \wedge \phi_{\text{softlock}}(D^*)]. \quad (3.26)$$

Ở đây ϕ_{guided} kiểm tra rằng đường tiến trình khả thi trong đồ thị nhiệm vụ thật sự đi qua các phòng đã ghép và còn traversable ở mức room-level; ϕ_{prog} kiểm tra tiến trình khóa–công–vật phẩm theo đồ thị nhiệm vụ; còn ϕ_{softlock} loại các trường hợp tồn tại ngõ cụt không hồi phục. Phép kiểm A* nguyên khối trên lưới ghép vẫn được ghi nhận riêng

như một kiểm tra nghiêm ngặt hơn, nhưng không còn là công pass/fail duy nhất của báo cáo. Cách viết này phản ánh đúng kiến trúc graph-first hybrid hiện tại và tránh đánh đồng “A* timeout trên lưới ghép” với “dungeon không chơi được”.

P-CBS như bộ xác nhận hành vi giới hạn nhận thức. Khác với procedural persona kiểu MCTS và các mô hình đường đi có bản đồ nhận thức chủ yếu mô phỏng định tính sự quên/nhầm hướng [41, 43, 44], P-CBS được hiện thực như một tìm kiếm heuristic trên không gian trạng thái tăng cường bởi bộ nhớ làm việc, độ tin cậy tri giác và ngân sách cân nhắc. Đóng góp được luận văn bảo vệ ở đây là một *validator persona-driven tích hợp vào pipeline sinh dungeon*, không phải một họ thuật toán tối ưu tổng quát mới nhằm thay thế A* trong mọi miền bài toán. Với một hành động a , hàm tiện ích dùng để xếp hạng nút kế là:

$$\begin{aligned}
 U(a) = & \alpha \Delta g + \beta \text{info_gain} - \gamma \text{risk} - \rho \text{revisit} \\
 & - \psi \text{complexity} - \omega \text{conditional_uncertainty} + \eta \text{frontier} \\
 & + \xi \text{affordance_resume} - \zeta \text{affordance_forgetting}.
 \end{aligned} \tag{3.27}$$

Hai hạng tử cốt lõi làm nên khác biệt của P-CBS là ρ và ω : revisit phạt việc quay lại ô đã đi qua dưới điều kiện bộ nhớ làm việc hữu hạn, còn conditional_uncertainty phạt việc đi vào các cấu trúc có điều kiện như cửa khóa, bombable gate, puzzle marker khi niềm tin cục bộ và inventory chưa đủ mạnh. Cách mô hình hóa này gần với khung bounded/resource-rational planning ở mức ý tưởng [45, 46], nhưng được hiện thực trực tiếp ở tầng quyết định node expansion của bộ giải dungeon thay vì ở tầng policy học sâu hay meta-reasoning tổng quát.

Persona và chỉ số nhận thức. P-CBS cho phép thay trọng số theo persona mà không thay đổi hợp đồng đầu vào của validator. Ví dụ, *speedrunner* dùng trọng số thấp cho ρ, ψ, ω và ưu tiên Δg ; trong khi *novice* dùng trọng số cao hơn cho nguy cơ chiến đấu, độ phức

tạp và bất định có điều kiện. Từ quỹ đạo đã sinh, hệ thống trích xuất các chỉ số hành vi như navigation entropy, tổng số revisits, và *aha-latency* (độ trễ trước khi persona vượt được vùng puzzle sau chuỗi do dự/lạc hướng):

$$H_{\text{nav}} = - \sum_{v \in \mathcal{V}_{\text{path}}} p(v) \log_2 p(v), \quad \text{CGR} = \frac{\sum_{j=1}^M \mathbf{1}[\phi_{\text{hard}}(D_j^*) = 1 \wedge \phi_{\text{pcbs}}(D_j^*) = 0]}{\sum_{j=1}^M \mathbf{1}[\phi_{\text{hard}}(D_j^*) = 1]}, \quad (3.28)$$

trong đó H_{nav} là entropy điều hướng và CGR là *cognitive gap rate*. Với thiết kế này, Chương 4 có thể đánh giá riêng: dungeon có đúng cơ học hay không, và nếu đúng cơ học thì persona nào còn bị lạc, sợ rủi ro hay bị chậm ở các vùng puzzle.

3.4 Thuật toán huấn luyện và suy luận trong mã nguồn

3.4.1 Huấn luyện theo giai đoạn

Huấn luyện được tổ chức theo chuỗi VQ-VAE $\rightarrow$ Diffusion $\rightarrow$ Fast Sampler $\rightarrow$ Masked Room, mỗi giai đoạn có điểm kiểm tra riêng để phục vụ tiếp tục huấn luyện và phân tích loại trừ. Trong giai đoạn diffusion, mỗi batch được xử lý theo các bước: mã hóa room thật sang latent thật, dựng điều kiện từ đồ thị sang phòng đúng lược đồ, tính $\mathcal{L}_{\text{diff}}$ với trọng số Min-SNR, sau đó cộng logic loss theo lịch warmup. Các cơ chế an toàn như bỏ qua batch không hữu hạn, cắt gradient và EMA được duy trì trong vòng lặp chính để hạn chế lỗi tối ưu hóa lan truyền theo thời gian huấn luyện.

Ở mức hiện thực, bộ lập lịch và điểm kiểm tra theo giai đoạn giữ vai trò neo để chuyển pha huấn luyện mà không làm đứt mạch tối ưu hóa. Khi kết hợp với kiểm tra không hữu hạn, cắt gradient và EMA, vòng lặp huấn luyện giữ được cả độ ổn định số học lẫn khả năng truy vết khi tiếp tục thí nghiệm.

Để mã giả có tính học thuật và dễ đối chiếu, luận văn theo quy ước trình bày thường gặp trong các bài báo gốc về diffusion và graph diffusion: tách rõ nhánh train/sampling, nêu tường minh input-output, và đặt tên bước theo hành động thuật toán ở mức trừu

tượng (ví dụ: lấy mẫu, gom batch, cập nhật) thay vì dùng trực tiếp tên hàm triển khai trong mã nguồn [11, 12, 34, 24].

Thuật toán 3 Huấn luyện một epoch diffusion với warmup logic

Đầu vào: Dataloader $\mathcal{D}$ trả về $(x, \text{graph_list})$, cấu hình huấn luyện

Đầu ra: Tham số diffusion/condition encoder được cập nhật ổn định

```

1:  $include\_logic \leftarrow (epoch \geq e_{warmup})$ 
2: Với mỗi  $(x, \text{graph\_list}) \in \mathcal{D}$  thực hiện
3:   Mã hóa từng graph:  $c_i \leftarrow \text{ENCODEGRAPHCONDITIONING}(\text{graph\_list}[i])$ 
4:   Gom các vector điều kiện thành  $C_{batch}$ 
5:   Gom các đồ thị thành  $G_{batch}$ 
6:   Tính  $\mathcal{L}_{diff}$  với Min-SNR weighting
7:   Nếu  $include\_logic$  thì
8:     Tính  $\mathcal{L}_{logic}$  trên predicted latent
9:   Ngược lại
10:     $\mathcal{L}_{logic} \leftarrow 0$ 
11:   end Nếu
12:   Nếu semantic supervision cho puzzle stage được bật thì
13:     Giải mã logits phòng dự đoán và tính  $\mathcal{L}_{stage}$ 
14:   Ngược lại
15:     $\mathcal{L}_{stage} \leftarrow 0$ 
16:   end Nếu
17:    $\mathcal{L}_{total} \leftarrow \alpha_{visual}\mathcal{L}_{diff} + \alpha_{logic}\mathcal{L}_{logic} + \lambda_{stage}\mathcal{L}_{stage}$ 
18:   Nếu  $\mathcal{L}_{total}$  hoặc gradient không hữu hạn thì
19:     Bỏ qua batch hiện tại, tăng bộ đếm non-finite, tiếp tục batch kế
20:   continue
21:   end Nếu
22:   Gradient clipping, OPTIMIZERSTEP, UPDATEEMA
23: end Đối với mỗi
24: Cập nhật scheduler và nhiệt độ LogicNet theo global step

```

Theo nghĩa phương pháp luận, Thuật toán 3 hiện thực trực tiếp nguyên tắc “ổn định trước, ràng buộc sau”: tín hiệu logic được đưa vào có kiểm soát để tránh làm lệch gradient ở giai đoạn mô hình chưa hội tụ đủ.

Trong mã nguồn, hai bước gom lô điều kiện và gom lô đồ thị được hiện thực bằng các hàm hỗ trợ đóng gói tensor tương ứng trong luồng huấn luyện diffusion.

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Ở bước cập nhật tham số, tính đúng được bảo đảm khi vòng lặp huấn luyện duy trì đồng thời bốn bất biến vận hành: hàm mất

mát và gradient luôn hữu hạn trên từng mini-batch, tensor điều kiện graph-to-room luôn tuân thủ đúng schema dữ liệu đã đặc tả, thành phần $\mathcal{L}_{\text{logic}}$ chỉ được kích hoạt sau ngưỡng warmup e_{warmup} , và thành phần $\mathcal{L}_{\text{stage}}$ chỉ được cộng khi batch có metadata stage hợp lệ. Quá trình lặp dừng một cách tự nhiên khi đã duyệt hết dataloader trong một epoch, tương đương $|\mathcal{D}|$ bước cập nhật. Với mô hình chi phí theo batch, độ phức tạp xấp xỉ:

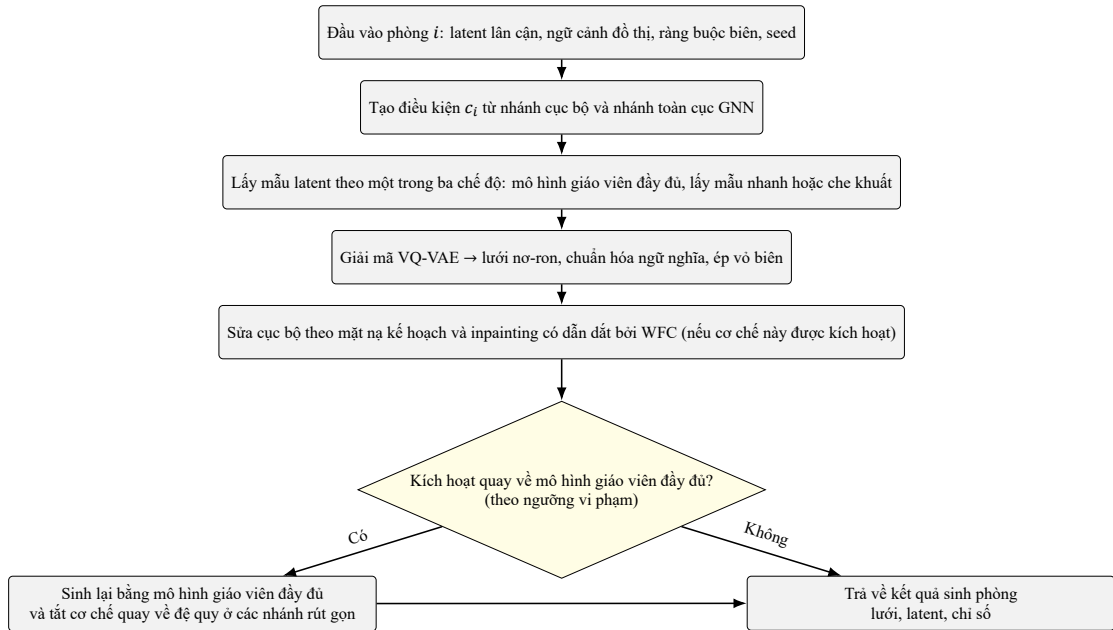
$$\mathcal{O}(|\mathcal{D}| \cdot (C_{\text{enc}} + C_{\text{diff}} + \mathbb{I}_{\text{logic}} C_{\text{logic}} + \mathbb{I}_{\text{stage}} C_{\text{stage}})), \quad (3.29)$$

trong đó C_{diff} là hạng chi phối.

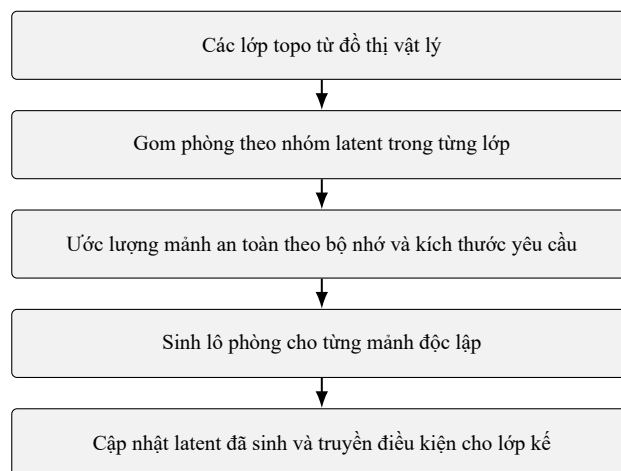
3.4.2 Suy luận nhiều phòng và cơ chế fallback

Trong luận văn này, mô hình giáo viên (teacher) là mô hình diffusion đầy đủ, được huấn luyện ở chế độ chuẩn và giữ vai trò nguồn tham chiếu có chất lượng cao nhất; các nhánh fast và masked là các biến thể suy luận rút gọn được chưng cất từ mô hình này để giảm chi phí. Fallback được hiểu là cơ chế an toàn tự động chuyển từ các nhánh sinh rút gọn sang mô hình giáo viên khi số chỉ số vi phạm vượt ngưỡng thiết kế, nhằm tái sinh lại phòng với cấu hình ổn định hơn. Quy trình suy luận dungeon được tổ chức theo ba mức: mức đồ thị, mức lớp phụ thuộc và mức phòng. Ở mức đồ thị, hệ thống chuẩn hóa đồ thị nhiệm vụ và dựng ngữ cảnh toàn cục. Ở mức lớp phụ thuộc, các phòng được sắp theo độ sâu topo [37] rồi gom theo nhóm latent để vừa giữ quan hệ phụ thuộc, vừa khai thác song song hóa. Ở mức phòng, điều kiện được tạo từ các phòng lân cận đã sinh; latent được lấy mẫu theo một trong ba chế độ thực thi, sau đó giải mã qua VQ-VAE và được hiệu chỉnh ký hiệu trước khi trả kết quả.

Bài toán sinh đa phòng được mô hình hóa như bài toán lập lịch có phụ thuộc thứ tự, do đó thuật toán tách xử lý theo lớp topo để bảo toàn quan hệ tiền nhiệm-hậu nhiệm. Trong từng lớp, các phòng được gom theo dạng latent và chia mảnh an toàn theo giới hạn bộ nhớ để tăng mức song song hóa mà không vi phạm ràng buộc logic. Nhánh tuần tự cho kích thước không chuẩn bảo đảm tính tương thích của quy trình.



Hình 3.2: Luồng nội bộ của quy trình sinh phòng, bao gồm bước hiệu chỉnh ký hiệu và cơ chế quay về mô hình giáo viên.



Hình 3.3: Luồng lập lịch sinh phòng theo lớp phụ thuộc và nhóm latent.

Thuật toán 4 Lập lịch sinh phòng theo lớp phụ thuộc

Đầu vào: Đầu vào đã chuẩn bị (G_{phys}, C) , cấu hình sinh phòng

Đầu ra: Tập kết quả phòng $\mathcal{R}$, latent phòng và chẩn đoán batch

```

1: Nếu chế độ batch phòng độc lập được kích hoạt và  $G_{phys}$  có hướng thì
2:    $\mathcal{L} \leftarrow \text{TOPOLOGICALGENERATIONLAYERS}(G_{phys})$ 
3:   Với mỗi layer  $\ell \in \mathcal{L}$  thực hiện
4:      $\mathcal{B} \leftarrow \text{BUCKETBYLATENTSHAPE}(\ell)$ 
5:     Với mỗi bucket  $b \in \mathcal{B}$  thực hiện
6:       Nếu kích thước phòng không chuẩn (16, 11) thì
7:         Sinh tuần tự từng phòng trong  $b$ 
8:       Ngược lại
9:          $k \leftarrow \text{ESTIMATESAFEBATCHSIZE}(b, k_{\max})$ 
10:        Với mỗi chunk độc lập kích thước  $k$  trong  $b$  thực hiện
11:           $\mathcal{R}_{chunk} \leftarrow \text{GENERATEROOMBATCH}(chunk, C)$ 
12:          Cập nhật  $\mathcal{R}$  và latent đã sinh
13:        end Đối với mỗi
14:      end Nếu
15:    end Đối với mỗi
16:  end Đối với mỗi
17: Ngược lại
18:   Sinh tuần tự theo thứ tự nút ổn định
19: end Nếu
20: Trả về:  $\mathcal{R}$ , latent phòng, chẩn đoán batch

```

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Đối với lập lịch đa phòng, tính đúng được bảo đảm vì trật tự lớp topo bảo toàn phụ thuộc tiên nhiệm $\rightarrow$ hậu nhiệm, cơ chế gom theo dạng latent tránh trộn lô khác hình dạng, và các phòng không đúng kích thước chuẩn được chuyển sang nhánh tuần tự để giữ hợp đồng tensor. Thuật toán dừng khi đã duyệt hết tập lớp, nhóm và mảnh trong mỗi nhóm. Vì vậy, chi phí phần lập lịch là:

$$\mathcal{O}(n + m) + \mathcal{O}(N), \quad (3.30)$$

và tổng chi phí sinh phòng là:

$$\sum_{b \in \mathcal{B}} \left\lceil \frac{|b|}{k_b} \right\rceil C_{\text{batch}}(k_b), \quad (3.31)$$

với k_b là safe chunk size của bucket b .

Thuật toán 5 Suy luận dungeon theo cơ chế graph-first

Đầu vào: Đồ thị nhiệm vụ tùy chọn G , siêu tham số suy luận Θ

Đầu ra: Dungeon D , tập phòng $\mathcal{R}$, metrics $\mathcal{M}$

- 1: **Nếu** $G = \emptyset$ và chế độ sinh cấu trúc đồ thị được bật **thì**
 - 2: $G \leftarrow \text{EVOLUTIONARYMISSIONGRAPHGENERATOR}(\Theta_{\text{graph}})$
 - 3: **end Nếu**
 - 4: $G_{\text{phys}} \leftarrow \text{FILTERVIRTUALNODES}(G)$
 - 5: $\mathcal{C} \leftarrow \text{PREPAREGRAPHCONTEXT}(G_{\text{phys}})$
 - 6: Chia các phòng theo lớp topo và nhóm latent
 - 7: **Với mỗi** bucket phòng độc lập **thực hiện**
 - 8: Sinh batch phòng bằng $\text{GENERATEROOMBATCH}(\mathcal{C}, \Theta)$
 - 9: Cập nhật latent đã sinh để cập điều kiện cho lớp kế
 - 10: **end Đối với mỗi**
 - 11: $D \leftarrow \text{STITCHROOMS}(\mathcal{R}, G_{\text{phys}})$
 - 12: **Nếu** bật đánh giá MAP-Elites **thì**
 - 13: $\mathcal{Q} \leftarrow \text{EVALUATEGENERATEDDUNGEON}(D)$
 - 14: **end Nếu**
 - 15: **Trả về:** $(D, \mathcal{R}, \mathcal{M})$
-

Thuật toán điều phối toàn bộ pipeline graph-first từ khâu chuẩn bị đồ thị đến ghép dungeon và đánh giá chất lượng. Cách tổ chức theo các pha có hợp đồng dữ liệu rõ ràng giúp kiểm soát sai lệch xuyên tầng, đồng thời hỗ trợ ablation và kiểm thử thành phần một

cách hệ thống. Vì vậy, tính đúng đắn được bảo toàn ở cả cấp phòng lẫn cấp dungeon.

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Ở cấp pipeline, tính đúng đòi hỏi đồ thị sinh mới (khi không có đầu vào) phải qua kiểm tra cấu trúc đồ thị trước khi sử dụng, các phòng phải được sinh theo lớp phụ thuộc và ghép trên đồ thị vật lý đã lọc virtual node, đồng thời các chỉ số hậu kỳ phải tách rõ tầng phòng và tầng dungeon. Quy trình dừng sau khi hoàn tất hữu hạn các pha chuẩn bị đồ thị, sinh phòng theo bucket/chunk, ghép dungeon và (tùy chọn) đánh giá QD. Theo đó, độ phức tạp tổng thể có thể viết:

$$\mathcal{O}(C_{\text{graph}} + C_{\text{rooms}} + C_{\text{stitch}} + C_{\text{eval}}), \quad (3.32)$$

trong đó C_{graph} được cho bởi Thuật toán 1.

Luồng suy luận một phòng và cơ chế quay về mô hình giáo viên

Ở mức vi mô, một phòng được xử lý theo chuỗi dựng điều kiện → lấy mẫu latent → giải mã → sửa cục bộ. Cơ chế quay về mô hình giáo viên đóng vai trò lớp an toàn, cho phép chuyển từ nhánh lấy mẫu nhanh sang nhánh chuẩn khi chỉ số vi phạm vượt ngưỡng thiết kế.

Cơ chế này cân bằng chi phí suy luận và độ tin cậy cấu trúc bằng cách cho phép các nhánh lấy mẫu nhanh hoạt động dưới giám sát hậu kiểm. Phòng được sinh theo chuỗi điều kiện → lấy mẫu latent → giải mã → sửa cục bộ; sau đó bộ chỉ số vi phạm quyết định có kích hoạt cơ chế quay về mô hình giáo viên hay không. Thiết kế hậu kiểm này bảo đảm các trường hợp vượt ngưỡng rủi ro được tái sinh bằng cấu hình an toàn hơn.

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Ở cấp một phòng, tính đúng được duy trì vì đầu ra luôn đi qua hậu kiểm tra biên/sửa lỗi, còn cơ chế quay về mô hình giáo viên chỉ kích hoạt khi chỉ số vi phạm vượt ngưỡng và khi đó chuyển sang nhánh an toàn hơn để khôi phục độ tin cậy. Quy trình dừng sau một lượt sinh chính và tối đa một lượt thử lại bằng mô hình giáo viên (không đệ quy), nên số vòng lặp luôn hữu hạn. Do đó, độ

Thuật toán 6 Sinh một phòng với cơ chế quay về mô hình giáo viên

Đầu vào: Điều kiện phòng $(z_N, z_S, z_E, z_W, b_i, p_i, G_i)$, cấu hình sampler

Đầu ra: Kết quả sinh phòng cho phòng i

- 1: $c_i \leftarrow \text{COMPUTEROOMCONDITION}(z, b_i, p_i, G_i)$
 - 2: **Nếu** chế độ suy luận là masked rồi rạc **thì**
 - 3: $z_i, \ell_i \leftarrow \text{MASKEDROOMSAMPLE}(c_i, G_i)$
 - 4: **Ngược lại Nếu** chế độ suy luận là categorical **thì**
 - 5: $z_i, \ell_i \leftarrow \text{CATEGORICALCODEBOOKSAMPLE}(c_i)$
 - 6: **Ngược lại**
 - 7: $z_i \leftarrow \text{DIFFUSIONSAMPLE}(c_i, G_i, \text{CFG}, \text{logic guidance})$
 - 8: $\ell_i \leftarrow \text{VQVAE.DECODE}(z_i)$
 - 9: **end Nếu**
 - 10: $x_i^{\text{neural}} \leftarrow \arg \max \ell_i$
 - 11: $x_i^{\text{final}} \leftarrow \text{REPAIRANDBOUNDARYENFORCE}(x_i^{\text{neural}}, G_i)$
 - 12: **Nếu** cần quay về mô hình giáo viên **thì**
 - 13: Sinh lại bằng mô hình giáo viên đầy đủ và vô hiệu hóa cơ chế quay về lồng nhau
 - 14: **end Nếu**
 - 15: **Trả về:** kết quả $(x_i^{\text{final}}, z_i, \text{metrics})$
-

phức tạp theo phòng là:

$$\mathcal{O}(C_{\text{sample}} + C_{\text{decode}} + C_{\text{repair}} + \mathbb{I}_{\text{fb}} C_{\text{teacher}}). \quad (3.33)$$

Để hạn chế rủi ro nhiều cấu trúc từ các nhánh suy luận chi phí thấp hơn mô hình giáo viên, quy trình áp dụng một luật quay về tường minh. Với một phòng r , cờ quay về được viết gọn như sau:

$$\begin{aligned} n_{\text{island}} \geq 1 \vee n_{\text{door-err}} \geq 1 \\ \mathbb{I}_{\text{fallback}}(r) = \mathbf{1} \left[\vee (n_{\text{repair}} \geq 18 \wedge n_{\text{obs}} > 0) \right. \\ \left. \vee (m = \text{masked} \wedge (r_{\text{overwrite}} > 0.34 \vee n_{\text{repair}} \geq 12)) \right]. \end{aligned} \quad (3.34)$$

Khi điều kiện này đúng, phòng được sinh lại bằng mô hình diffusion đầy đủ ở vai trò teacher. Cơ chế này cho thấy hệ thống ưu tiên độ tin cậy của đầu ra hơn lợi ích tốc độ cục bộ của từng bộ lấy mẫu.

3.4.3 Đánh giá sau sinh và chỉ số vận hành

Sau khi ghép phòng, hệ thống không đưa trực tiếp lưới ghép vào bộ giải mà luôn đi qua một *validation handoff contract*. Cụ thể, thủ tục `PrepareValidationGrid` chuẩn hóa shape, ép semantic ID về palette chuẩn, loại VOID kín và chuẩn hóa duy nhất một cặp start-goal. Chỉ sau bước này dungeon mới được chuyển sang hai tầng kiểm chứng sau sinh: oracle cứng và P-CBS. MAP-Elites và LogicNet vẫn được giữ lại, nhưng với vai trò hỗ trợ: MAP-Elites đo chất lượng–đa dạng ở mức dungeon, còn LogicNet đóng vai trò tín hiệu chặn đoán cấp phòng thay vì quyết định cuối cùng về tính playable.

Các chỉ số tổng hợp vận hành được ghi trực tiếp ở cuối đường sinh dungeon, trong đó có:

$$\text{repair_rate} = \frac{1}{N} \sum_{i=1}^N r_i, \quad \text{hard_solvable_rate} = \frac{1}{M} \sum_{j=1}^M \phi_{\text{hard}}(D_j^*), \quad (3.35)$$

$$\text{pcbs_success_rate} = \frac{1}{M} \sum_{j=1}^M \phi_{\text{pcbs}}(D_j^*), \quad \text{CGR} = \frac{\sum_{j=1}^M \mathbf{1}[\phi_{\text{hard}}(D_j^*) = 1 \wedge \phi_{\text{pcbs}}(D_j^*) = 0]}{\sum_{j=1}^M \mathbf{1}[\phi_{\text{hard}}(D_j^*) = 1]}. \quad (3.36)$$

Ở đây $r_i = 1$ nếu phòng i có sửa lỗi; ϕ_{hard} là phán quyết của oracle cơ học; còn ϕ_{pcbs} là kết quả của persona P-CBS đã chọn. Việc tách chỉ số như trên giúp phân biệt ba lớp lỗi: lỗi hình học phòng, lỗi tiến trình cơ học của toàn dungeon, và lỗi ma sát nhận thức chỉ lộ ra khi dùng persona giới hạn nhận thức.

Đánh giá hậu sinh vì vậy được phân tầng theo đúng vai trò của từng thành phần. Oracle cứng trả lời câu hỏi “dungeon có đúng cơ học hay không”; P-CBS trả lời câu hỏi “dungeon này gây nhầm lẫn hay chậm trễ cho persona nào”; MAP-Elites trả lời câu hỏi “nghiệm đó nằm ở đâu trong không gian chất lượng–đa dạng”; còn LogicNet tiếp tục đóng vai trò chặn đoán nội bộ ở mức phòng. Cách tách này giúp các ablation ở Chương 4 không còn trộn lẫn lỗi cơ học với lỗi hành vi.

Thuật toán 7 Đánh giá dungeon sau ghép với hợp đồng validation, oracle cứng và P-CBS

Đầu vào: Dungeon đã ghép D , tập phòng $\mathcal{R}$, đồ thị vật lý G_{phys} , tập persona $\mathcal{P}$

Đầu ra: Phán quyết oracle, chỉ số P-CBS, chỉ số QD và danh sách phòng lỗi

```

1:  $D^*, info_{handoff} \leftarrow \text{PREPAREVALIDATIONGRID}(D)$ 
2:  $s_{hard} \leftarrow \text{HARDORACLE}(D^*, G_{phys})$ 
3: Nếu  $s_{hard}$  giải được thì
4:   Với mỗi  $p \in \mathcal{P}$  thực hiện
5:      $m_p \leftarrow \text{RUNPCBS}(D^*, p)$ 
6:   end Đối với mỗi
7:   Nếu bật MAP-Elites và thành phần MAP-Elites khả dụng thì
8:      $\text{MAPELITESADDDUNGEON}(D^*, s_{hard}, G_{phys})$ 
9:     Trích xuất các chỉ số QD từ  $D^*$ 
10:  end Nếu
11: end Nếu
12: Nếu LogicNet khả dụng thì
13:   Với mỗi phòng  $r_i \in \mathcal{R}$  có latent thực hiện
14:      $(\ell_i, info_i) \leftarrow \text{LOGICNET}(z_i)$ 
15:     Cộng dồn chỉ số chẩn đoán và ghi phòng lỗi nếu dưới ngưỡng
16:   end Đối với mỗi
17: end Nếu
18: Trả về:  $info_{handoff}, s_{hard}, \{m_p\}_{p \in \mathcal{P}}$  cùng các chỉ số đánh giá và danh sách phòng lỗi

```

Tiêu chí tính đúng, điều kiện dừng và độ phức tạp. Ở tầng đánh giá hậu sinh, dungeon chỉ được đưa vào archive sau khi oracle cứng xác nhận khả giải; P-CBS chỉ chạy trên lưới đã qua hợp đồng validation để tránh nhiễu do token sai chuẩn; còn LogicNet được cộng dồn trên tập phòng có latent như một kênh chẩn đoán riêng. Quy trình dừng khi hoàn tất chuỗi hữu hạn gồm chuẩn hóa lưới ghép, oracle cứng, duyệt persona và duyệt danh sách phòng có latent. Vì vậy, độ phức tạp là:

$$\mathcal{O}(C_{handoff}(D) + C_{hard}(D, G) + |\mathcal{P}| C_{pcbs}(D) + N_z C_{logicnet} + C_{archive}). \quad (3.37)$$

3.4.4 Giả định vận hành, chế độ lỗi và biện pháp giảm thiểu

Giả định vận hành. Mô tả phương pháp trong chương này dựa trên ba giả định vận hành cơ bản: (i) đồ thị vật lý sau khi lọc nút ảo vẫn liên thông và có cặp start-goal hợp lệ, (ii) phòng đầu vào/đầu ra tuân thủ hợp đồng kích thước và từ vựng tile, và (iii) lưới ghép trước validator luôn đi qua bước chuẩn hóa validation handoff để loại token ngoài bảng

từ điển và chuẩn hóa terminal. Viết gọn:

$$\text{Connected}(G_{\text{phys}}) = 1, \quad s, g \in V_{G_{\text{phys}}}, s \neq g, \quad (3.38)$$

$$x_i \in \{0, \dots, 43\}^{16 \times 11}, \quad r_d \leq h_d \ (d \in \{N, S, E, W\}). \quad (3.39)$$

Các giả định này không thay cho đánh giá thực nghiệm, nhưng giúp xác định rõ miền hiệu lực của các thuật toán ở Mục 3.4.

Chế độ lỗi và giảm thiểu. Bảng 3.3 tóm tắt các lỗi vận hành thường gặp và cơ chế giảm thiểu tương ứng trong pipeline.

Bảng 3.3: Các chế độ lỗi chính và cơ chế giảm thiểu trong đường chạy Chương 3

Nhóm lỗi	Biểu hiện quan sát được	Cơ chế giảm thiểu trong pipeline
Sai schema tensor	Lỗi shape ở conditioning hoặc batch diffusion.	Fail-fast tại điểm vào khối B_3/B_4 , kiểm tra kiểu-kích thước trước khi lấy mẫu.
Đồ thị không khả thi	Đồ thị vi phạm ràng buộc tiến trình hoặc không đạt điều kiện hậu kiểm.	Chọn nghiệm khả thi-ưu tiên, phạt vi phạm mềm, hậu kiểm tra trước khi ghép phòng.
Nhiều cấu trúc từ bộ lấy mẫu nhanh	Xuất hiện island, lỗi cửa, hoặc overwrite cao ở chế độ masked.	Luật quay về mô hình giáo viên theo ngưỡng vi phạm, đồng thời chặn quay về đệ quy.
Lệch biên-cửa sau giải mã	Cửa bắt buộc mất hoặc biên ngoài không kín.	Ép lớp biên, sửa theo mặt nạ kế hoạch, inpainting có dẫn dắt bởi WFC.
Rò rỉ VOID/token ngoài bảng từ điển	Lưới ghép chứa ô không hợp lệ hoặc VOID kín làm sai phán quyết validator.	Bắt buộc chạy PrepareValidationGrid: ánh xạ lại semantic ID, lấp VOID kín, chuẩn hóa đúng một start-goal.
Suy giảm thành phần phụ trợ	LogicNet, MAP-Elites hoặc nhánh P-CBS tạm không khả dụng ở một số cấu hình ablation.	Chạy suy giảm có kiểm soát: giữ oracle cứng làm chuẩn tối thiểu, tách rõ chỉ số nào bị vô hiệu hóa.

3.5 Cấu hình triển khai và khả năng tái lập

Cấu hình chuẩn được quản lý tập trung trong bộ tham số thí nghiệm, trong đó các tham số lỗi như kích thước phòng, số lớp tile, số chiều latent, hệ số CFG, số bước diffusion và cờ bật/tắt repair đều được khai báo tường minh. Cơ chế seed được đồng bộ cho các thành phần sinh ngẫu nhiên chính của hệ thống; trong thiết lập phân tán, mỗi worker

dùng seed lệch theo chỉ số tiến trình để tách luồng ngẫu nhiên nhưng vẫn bảo toàn khả năng truy vết. Hệ điểm kiểm tra được tổ chức theo giai đoạn, lưu kèm siêu dữ liệu kiến trúc và trạng thái huấn luyện, đồng thời dùng cơ chế lưu an toàn để giảm nguy cơ hỏng điểm kiểm tra trong các phiên chạy dài.

Về tiêu chí dừng, tầng cấu trúc đồ thị nhiệm vụ có thể dừng sớm khi fitness hội tụ; tầng diffusion dừng theo số epoch cấu hình và chọn điểm kiểm tra theo chỉ số xác thực; tầng suy luận dừng theo số phòng cùng giới hạn số vòng sửa lỗi. Cách tổ chức này tạo ra đường chạy nhất quán giữa huấn luyện và suy luận, đồng thời giữ chi phí tính toán trong ngưỡng phù hợp với quy mô dữ liệu hiện có.

3.6 Ánh xạ nội dung phương pháp vào luồng hiện thực

Để bảo đảm Chương 3 bám sát trật tự vận hành của hệ thống, các nội dung được ánh xạ theo bốn cụm chức năng: Mục 3.2.1–3.2.3 làm rõ hai lớp đồ thị và ma trận hợp đồng khối; Mục 3.2.4–3.2.7 chốt luồng dữ liệu, tiền xử lý và bất biến vận hành; Mục 3.3 trình bày nguyên tắc chọn thiết kế và đặc tả thuật toán từng khối; Mục 3.4 mô tả quy trình huấn luyện/suy luận kèm giả định vận hành và chế độ lỗi; Mục 3.5 tổng hợp cấu hình triển khai và tái lập. Cách phân tách này giúp người đọc lần theo mạch từ lập luận thiết kế đến hành vi thực thi mà không phải chuyển ngữ cảnh đột ngột giữa các tầng.

Ở mức kiểm chứng, cấu trúc này cũng tạo điều kiện cho phân tích loại trừ theo từng khối: tiên nghiệm cấu trúc đồ thị (Khối I), điều kiện hóa (Khối III), diffusion (Khối IV), hướng dẫn logic (Khối V), sửa lỗi ký hiệu (Khối VI) và lớp xác nhận hậu sinh gồm oracle cứng, P-CBS và QD (Khối VII). Nhờ đó, phân phương pháp vừa giữ được tính diễn giải học thuật, vừa duy trì tính khả kiểm trong mã nguồn.

3.7 Liên kết với thiết kế thực nghiệm Chương 4

Thiết kế phương pháp trong chương này chuyển thẳng thành các trục kiểm chứng của Chương 4. Vì mỗi khối có giao diện rõ ràng và cờ bật/tắt độc lập, thí nghiệm phân tích loại trừ có thể ước lượng minh bạch đóng góp biên của hướng dẫn logic, tiên nghiệm cấu trúc đồ thị, sửa lỗi ký hiệu, cơ chế quay về mô hình giáo viên và lớp xác nhận hành vi P-CBS. Việc đặt các nhánh giáo viên, nhanh và che khuất trong cùng một quy trình, kèm cơ chế kẹp guidance theo miền chung cất, giúp so sánh công bằng hơn dưới cùng ngân sách vận hành. Đồng thời, chuẩn hóa seed, điểm kiểm tra và ghi log cho phép triển khai đồng thời ba nhóm kiểm chứng: oracle cơ học, persona-benchmark và đánh giá OOD/thống kê theo cặp (bootstrap và FDR) với độ lặp lại cao [14, 15].

3.8 Tóm tắt chương

Chương 3 đã đặc tả đầy đủ quy trình no-ron-ký hiệu theo hướng lấy đồ thị làm trung tâm ở mức triển khai: từ biểu diễn dữ liệu, cơ chế điều kiện hóa từ đồ thị sang phòng, công thức huấn luyện/suy luận của diffusion, đến các lớp sửa lỗi, hợp đồng bàn giao validation và lớp xác nhận hậu sinh gồm oracle cứng, P-CBS và QD. Điểm cốt lõi của phương pháp là tách trách nhiệm theo tầng nhưng liên kết bằng một hợp đồng dữ liệu thống nhất; nhờ đó hệ thống vừa kiểm soát được tiến trình toàn cục, vừa duy trì chất lượng hình học ở mức cục bộ và đo được ma sát nhận thức ở mức persona. Trên nền đặc tả này, Chương 4 có cơ sở để đánh giá định lượng đóng góp của từng thành phần trong các thiết lập cùng ngân sách và OOD.

CHƯƠNG 4

Kết quả thực nghiệm và thảo luận

Chương này trình bày kế hoạch thực nghiệm và khung phân tích kết quả theo đúng các trục kiểm chứng đã thiết kế ở Chương 1 và Chương 3. Nội dung được tổ chức theo mạch: mục tiêu kiểm chứng → thiết kế giao thức → kết quả định lượng → thảo luận cơ chế và giới hạn.

4.1 Mục tiêu kiểm chứng và câu hỏi thực nghiệm

Phần này đặt lại các câu hỏi nghiên cứu ở dạng kiểm chứng thực nghiệm để làm rõ tiêu chí đánh giá thành công của phương pháp:

- (RQ1) Cấu hình nơ-ron–ký hiệu lấy đồ thị làm trung tâm (graph-first) có cải thiện đồng thời tính hợp lệ gameplay, khả năng giải được và chất lượng phòng so với các cấu hình đối chứng hay không?
- (RQ2) Đóng góp biên của từng khối (prior cấu trúc đồ thị, conditioning, diffusion, logic guidance, symbolic repair, cơ chế quay về) có ý nghĩa thống kê hay không?
- (RQ3) Khi bị ràng buộc cùng ngân sách đánh giá, phương pháp đề xuất có giữ lợi thế so với các cấu hình đối chứng sinh đồ thị nhiệm vụ hay không?
- (RQ4) Hiệu năng có duy trì ổn định trên các kịch bản OOD-small và OOD-large theo quy mô số phòng hay không?

Trong phần trình bày dưới đây, mỗi câu hỏi được gắn với một nhóm chỉ số riêng để tránh đánh giá cảm tính. RQ1 được kiểm tra bằng `feasible_rate`, `constraint_valid_rate`, `repair_rate`, `generation_time_sec` và các chỉ số khả giải; RQ2 dùng paired bootstrap và kiểm định hoán vị dấu theo cặp trên các metric chính; RQ3 đọc theo matched-budget

cùng số lần đánh giá; RQ4 xem chênh lệch khi chuyển từ ID sang OOD-small/OOD-large. Nhờ vậy, các kết luận ở cuối chương có thể truy ngược về đúng thước đo mà nó phụ thuộc.

4.2 Thiết kế thực nghiệm

4.2.1 Thiết lập tái lập và môi trường thực nghiệm

Mục này nêu chi tiết phần cứng/phần mềm, phiên bản thư viện, seed, chiến lược checkpoint và logging. Các cấu hình được cố định để bảo đảm mỗi thí nghiệm có thể chạy lại và đối chiếu trực tiếp.

Để bảo đảm tái lập, toàn bộ luồng đánh giá giữ nguyên seed, checkpoint và cấu hình sinh trong mỗi lần chạy. Mỗi mẫu sinh đều ghi nhận thời gian sinh end-to-end, số tile phải sửa, số marker đồ thị bị ghi đè và các chẩn đoán hậu kiểm, nhờ đó kết quả không chỉ phản ánh chất lượng đầu ra mà còn phản ánh chi phí vận hành thực. Với các so sánh nội bộ, topology và vị trí phòng được giữ cố định để giảm nhiễu do thay đổi đầu vào.

4.2.2 Dữ liệu và kịch bản đánh giá

Mục này mô tả tập huấn luyện/kiểm định và cách dựng các kịch bản đánh giá ID, OOD-small, OOD-large. Phần này cũng chốt nguyên tắc không rò rỉ dữ liệu giữa các split ở cấp mission graph.

Việc chia tập tuân thủ nguyên tắc rời nhau theo nguồn mission graph, nhờ đó các mẫu cùng xuất xứ không bao giờ rơi vào hai split khác nhau. Kịch bản ID phản ánh phân phối huấn luyện; OOD-small giảm số phòng hoặc độ sâu đường đi; OOD-large tăng quy mô và nhánh rẽ để đo khả năng giữ cấu trúc khi ngoại suy. Cách dựng kịch bản này giúp tách rõ lỗi do ghi nhớ dữ liệu và lỗi do yếu mô hình.

4.2.3 Baseline và cấu hình so sánh

Mục này liệt kê các baseline theo từng tầng (graph generation, room generation, repair), cùng các cấu hình ablation bật/tắt khối trong pipeline đề xuất để so sánh công bằng.

Ở tầng sinh mission graph, bộ so sánh chuẩn gồm RANDOM, ES, GA, MAP_ELITES và FULL; ở tầng sinh phòng, ba nhánh diffusion_cfg3_logic0_steps50, fast_cfg3_logic0_steps4 và masked_room_full cho phép tách riêng ảnh hưởng của số bước lấy mẫu và cơ chế hậu xử lý. Ở tầng sửa lỗi, ablation FULL / COND_NO_TPE / COND_WEAK_GRAPH cho thấy tác động của TPE và cross-attention trên cùng topology. Cách tổ chức này đảm bảo mỗi baseline trả lời một câu hỏi khác nhau thay vì chồng lấn trách nhiệm.

4.2.4 Chỉ số đánh giá và suy luận thống kê

Mục này đặc tả bộ chỉ số chính: hợp lệ, giải được, chất lượng, đa dạng, repair rate, fallback rate và chi phí suy luận. Suy luận thống kê dùng paired bootstrap, kiểm định hoán vị dấu theo cặp và hiệu chỉnh Benjamini-Hochberg FDR.

Các chỉ số được đọc theo bốn lớp: hợp lệ và khả giải, hình thái và tiến trình, đa dạng, và chi phí. Với chỉ số điều kiện theo loại cửa/cổng, khi topology không có loại gate tương ứng thì giá trị nên được hiểu là 1.0 có điều kiện, không phải là ưu thế mô hình. Phần đa dạng nên dựa trên coverage, QD-score, novelty và graph edit distance; phần chi phí phải tính end-to-end, vì thời gian sinh không chỉ nằm ở sampler mà còn ở repair, overlay và kiểm tra hậu kỳ.

4.2.5 Giao thức thí nghiệm

Mục này mô tả ba nhánh thí nghiệm lỗi:

- (a) Fixed-seed ablation theo khối chức năng.

(b) Matched-budget benchmark giữa các phương pháp sinh đồ thị nhiệm vụ.

(c) OOD scaling theo quy mô dungeon và số phòng.

Ba nhánh thí nghiệm lỗi bổ sung cho nhau. Fixed-seed ablation giữ topology và seed cố định để nhìn đúng ảnh hưởng của từng khối; matched-budget benchmark khóa số lần đánh giá để so sánh công bằng giữa các chiến lược search; OOD scaling thay đổi quy mô dungeon trong khi giữ nguyên miền logic để xem hệ có còn ổn định hay không. Mỗi nhánh nên được báo cáo với khoảng tin cậy bootstrap để tránh kết luận dựa vào một lần chạy đơn lẻ.

4.3 Kết quả định lượng

4.3.1 Kết quả huấn luyện VQ-VAE nền tảng

Sáu cấu hình VQ-VAE được huấn luyện trên cùng ngân sách 300 epoch để tách riêng ảnh hưởng của ba nhóm yếu tố: kích thước codebook, độ rộng hidden layer, và hai thành phần kiến trúc CoordConv/MRF. Khi đọc kết quả, cần phân biệt giữa hai khía cạnh: chất lượng hội tụ tại epoch tốt nhất và độ ổn định của mô hình về cuối quá trình huấn luyện. Bảng dưới đây tóm tắt các chỉ số này; với hai cột loss, giá trị càng thấp càng tốt, còn perplexity cao hơn cho thấy codebook được khai thác đa dạng hơn. Các giá trị tốt nhất theo từng cột được in đậm. Đồng thời, tất cả các run đều đạt codebook utilization 100%, nên khác biệt quan sát được chủ yếu phản ánh năng lực tối ưu hóa chứ không phải hiện tượng collapse của latent.

Bảng 4.1: So sánh sáu cấu hình VQ-VAE trên cùng lịch huấn luyện 300 epoch

Cấu hình	Epoch tốt nhất	Val loss tốt nhất	Val loss cuối	Perplexity tốt nhất	Sử dụng codebook
Cơ sở	294	6.93e-5	1.05e-4	40.48	100%
Codebook 128	161	8.82e-5	6.90e-4	35.89	100%
Codebook 512	241	2.52e-3	5.63e-3	42.34	100%
Hidden 64	119	7.47e-5	1.26e-4	40.79	100%
Bỏ CoordConv	203	3.19e-3	6.14e-3	39.95	100%
Bỏ MRF	187	2.34e-4	5.08e-3	40.57	100%

Kết quả cho thấy baseline là cấu hình cân bằng nhất: vừa đạt val loss thấp nhất, vừa giữ khoảng cách giữa giá trị tốt nhất và giá trị cuối ở mức nhỏ, nên quá trình hội tụ ổn định hơn so với các biến thể ablation. Hai biến thể hidden 64 và codebook 128 có chất lượng khá gần baseline nhưng không tạo ra cải thiện rõ rệt. Ngược lại, codebook 512 và đặc biệt là việc bỏ CoordConv làm chất lượng hội tụ giảm đáng kể; bỏ MRF cũng làm val loss cuối tăng lên rõ rệt, dù giá trị tốt nhất vẫn còn chấp nhận được. Từ góc nhìn này, CoordConv và MRF là hai thành phần có đóng góp rõ ràng nhất cho độ ổn định của tiền huấn luyện.

4.3.2 Kết quả tổng thể so với baseline

Trình bày bảng kết quả chính và so sánh định lượng giữa phương pháp đề xuất với các baseline theo từng nhóm chỉ số.

Trong so sánh topology thủ công có semantic content rõ và vị trí phòng cố định, diffusion_cfg3_logic0_steps50 đạt repair_rate = 1.000 với 378 tile được sửa trong 52.99 giây. fast_cfg3_logic0_steps4 vẫn đạt repair_rate = 1.000 nhưng mất 100.54 giây và sửa 584 tile, còn masked_room_full đạt repair_rate = 0.833 trong 137.83 giây với 581 tile được sửa. So sánh pairwise cho thấy fast so với diffusion tạo ra 130 tile thay đổi, còn masked so với fast tạo ra 146 tile thay đổi; các chuyển đổi structure_to_floor chiếm ưu thế. Trên bộ chạy này, nhánh diffusion là cấu hình cân bằng nhất giữa chi phí và mức

can thiệp, còn masked room là nhánh tốn kém và can thiệp sâu nhất.

4.3.3 Kết quả ablation theo khối pipeline

Phân tích mức suy giảm khi tắt từng thành phần để lượng hóa đóng góp biên và chỉ ra các khối ảnh hưởng mạnh nhất tới độ tin cậy gameplay.

Trong smoke test scale12, FULL, COND_NO_TPE và COND_WEAK_GRAPH đều giữ success_rate = 1.0 và constraint_valid_rate = 1.0, tức là việc làm yếu conditioning không phá vỡ tính hợp lệ nhị phân trên topology cố định này. Tuy nhiên, COND_WEAK_GRAPH làm generation_time_sec tăng từ 20.89 giây lên 39.18 giây, và kiểm định đi kèm cho thấy chênh lệch này có ý nghĩa sau hiệu chỉnh BH-FDR. Trước lớp overlay, mức khớp marker đồ thị chỉ đạt 0.7083 và sai số semantic anchor trung bình là 7.875; sau overlay, khớp marker trở về 1.0. Điều này cho thấy khối sửa ký hiệu không chỉ là bước làm đẹp đầu ra mà là lớp phục hồi ngữ nghĩa bắt buộc. Vì repair_rate trong smoke test này bằng 0.0, biến động quan sát được chủ yếu phản ánh chi phí và độ ổn định marker chứ không phải tải sửa tile.

4.3.4 Kết quả matched-budget ở tầng đồ thị nhiệm vụ

So sánh công bằng theo cùng ngân sách đánh giá để làm rõ ưu/nhược của GA, CVT/CMA-emitter và cấu hình đề xuất trong chất lượng-đa dạng.

Ở tầng mission graph, matched-budget comparison nên được đọc như một bài kiểm tra hiệu quả của chiến lược tìm kiếm khi số lần đánh giá được cố định. RANDOM đóng vai trò ngưỡng dưới, ES cho biết lợi ích của mutation-only search, GA bổ sung crossover để khai thác vùng lời giải, còn MAP_ELITES và FULL kiểm tra khả năng giữ coverage và chất lượng descriptor trong cùng ngân sách. Với protocol này, việc báo cáo không chỉ giá trị trung bình mà cả khoảng tin cậy và p-value là cần thiết, vì chất lượng-đa dạng thường thể hiện ở hình dạng phân bố chứ không chỉ ở một điểm tối ưu đơn lẻ.

4.3.5 Kết quả OOD theo quy mô dungeon

Báo cáo mức biến động chỉ số khi chuyển từ ID sang OOD-small/ODD-large, đồng thời chỉ ra thành phần nào giúp hệ giữ ổn định tốt hơn.

Khi chuyển sang OOD-small và OOD-large, điều cần quan sát không chỉ là điểm số giảm bao nhiêu mà là hình thái suy giảm. OOD-small thường làm lộ mức phụ thuộc vào dữ liệu huấn luyện ở các nhánh path, gate và branch, còn OOD-large thử khả năng duy trì hợp đồng dữ liệu khi số phòng và độ sâu tiến trình tăng. Một hệ thống thật sự ổn định sẽ dễ độ giải được và chỉ số hợp lệ giảm chậm hơn chi phí sinh tăng, đồng thời giữ sửa lỗi và quay về trong ngưỡng chấp nhận được. Vì vậy, OOD là nơi kiểm tra trực tiếp giá trị của phép tách lấy đồ thị làm trung tâm, thay vì chỉ kiểm tra năng lực tái tạo phòng.

4.3.6 Độ tin cậy vận hành và chi phí

Phân tích repair rate, fallback rate, lỗi hậu kiểm và thời gian suy luận để đánh giá đánh đổi giữa độ tin cậy và chi phí tính toán.

Độ tin cậy vận hành phải đọc cùng chi phí chứ không đọc tách rời. Ở bộ so sánh nội bộ, diffusion không chỉ nhanh nhất mà còn đạt repair_rate cao nhất, trong khi fast sampler không tự động nhanh hơn do phần repair và overlay chiếm tỉ trọng đáng kể; masked_room_full vừa chậm nhất vừa có repair_rate thấp hơn. Khoảng cách giữa pre-overlay và post-overlay graph-marker exact match từ 0.7083 lên 1.0 cho thấy lớp symbolic overlay là điểm khóa để giữ nhất quán ngữ nghĩa cuối cùng. Nói cách khác, kết quả đầu ra chỉ nên được xem là hoàn tất sau khi đi qua cả sửa tile, khôi phục marker và kiểm tra hợp lệ cuối.

4.4 Thảo luận

4.4.1 Giải thích cơ chế cải thiện

Liên hệ trực tiếp kết quả thực nghiệm với cơ chế phương pháp của Chương 3 để giải thích vì sao pipeline đạt cải thiện hoặc suy giảm trong từng thiết lập.

Những cải thiện quan sát được chủ yếu xuất phát từ việc tách rõ hai tầng trách nhiệm: tầng đồ thị giữ tiến trình, tầng phòng giữ hình học. Khi một nhánh làm yếu conditioning hoặc bỏ bớt tín hiệu graph, hệ vẫn còn có thể sinh ra mẫu hợp lệ trên topology cố định, nhưng thời gian tăng và lớp hậu xử lý phải làm việc nhiều hơn. Ngược lại, semantic overlay làm giảm sai lệch marker từ 29.2% xuống 0%, nên đây không phải bước tùy chọn mà là cơ chế phục hồi ngữ nghĩa.

4.4.2 Đánh đổi chất lượng, đa dạng và chi phí

Thảo luận các điểm đánh đổi khi thay đổi cấu hình bộ lấy mẫu, mức hướng dẫn logic và chính sách quay về, từ đó đề xuất cấu hình khuyến nghị cho từng mục tiêu sử dụng.

Điểm đánh đổi lớn nhất nằm ở chỗ số bước lấy mẫu thấp không đồng nghĩa với throughput cao hơn nếu repair và overlay chiếm ưu thế. Vì vậy, tốc độ nên được đo end-to-end trên toàn pipeline, không chỉ trên sampler. Trong cấu hình hiện tại, diffusion là điểm cân bằng tốt nhất; fast sampler phù hợp khi muốn rút ngắn pha sinh nhưng vẫn chấp nhận chi phí hậu xử lý; masked room nên được dùng như nhánh dự phòng khi cần inpainting có định hướng mạnh.

4.4.3 Mối đe dọa tính hiệu lực và giới hạn

Nêu các nguồn sai lệch tiềm tàng (độ nhỏ dữ liệu, phụ thuộc seed, giới hạn benchmark, phụ thuộc bộ ràng buộc) và phạm vi mà kết luận thực nghiệm có thể khái quát.

Hạn chế lớn nhất của bộ kết quả hiện có là phần lớn vẫn là kiểm chứng nội bộ trên topology cố định hoặc smoke test quy mô nhỏ. Điều đó giúp đọc cơ chế rất rõ nhưng

chưa đủ để khẳng định ưu thế tổng quát trên mọi miền topology. Ngoài ra, matched-budget ngoại sinh với các baseline công bố rộng hơn vẫn là bước cần làm trước khi đưa ra tuyên bố cạnh tranh ở mức SOTA. Kết luận ở chương này vì thế nên hiểu là xác nhận tính khả thi và độ ổn định cục bộ, không phải là đóng dấu cuối cùng cho toàn bộ hướng nghiên cứu.

4.5 Tóm tắt chương

Chương 4 tổng hợp kết quả kiểm chứng theo bốn trục: hiệu năng tổng thể, đóng góp thành phần, công bằng ngân sách và khả năng khái quát OOD. Các kết quả này là cơ sở trực tiếp cho phần kết luận, giới hạn và định hướng mở rộng ở Chương 5.

CHƯƠNG 5

Kết luận và hướng phát triển

Chương này tổng kết những đóng góp chính của luận văn, chỉ ra các giới hạn còn lại và nêu các hướng phát triển gần nhất. Cách trình bày đi từ kết quả đạt được sang các điểm còn mở để giữ ranh giới rõ giữa phần đã kiểm chứng và phần cần tiếp tục nghiên cứu.

5.1 Tổng kết đóng góp

Luận văn đã xây dựng một pipeline sinh dungeon theo hướng graph-first neural-symbolic, trong đó đồ thị nhiệm vụ giữ vai trò neo cho tiến trình còn mô hình sinh phòng đảm nhận hình học cục bộ. Điểm quan trọng không chỉ nằm ở việc tách hai tầng, mà còn ở cách hai tầng này được nối với nhau bằng điều kiện hóa hai luồng, khử nhiễu trong không gian latent, sửa lỗi ký hiệu và cơ chế quay về mô hình giáo viên khi đầu ra vượt ngưỡng rủi ro.

Trên các so sánh nội bộ đã có, cấu hình `diffusion_cfg3_logic0_steps50` cho thấy điểm cân bằng tốt giữa chi phí và mức can thiệp, còn symbolic overlay là bước bắt buộc để phục hồi marker đồ thị từ mức khớp trước overlay 0.7083 lên 1.0. Kết quả này củng cố luận điểm rằng chất lượng dungeon không nên được hiểu như đầu ra của một bộ sinh duy nhất, mà là kết quả của một chuỗi các lớp kiểm soát từ đồ thị đến phòng rồi đến hậu kiểm.

Về mặt phương pháp, luận văn cũng đã chuẩn hóa được cách đọc các chỉ số đánh giá theo từng tầng: hợp lệ và khả giải ở mức dungeon, độ ổn định và chi phí ở mức suy luận, cùng coverage và QD-score ở mức archive. Nhờ đó, phần thực nghiệm không còn bị dàn trải theo kiểu mô tả trực giác, mà có thể truy hồi về các câu hỏi nghiên cứu và các bất

biến hệ thống đã đặt ra từ đầu.

5.2 Hạn chế của nghiên cứu

Hạn chế rõ nhất của luận văn hiện tại là phần lớn kết quả vẫn nằm trong phạm vi kiểm chứng nội bộ, với một số topology thủ công hoặc smoke test quy mô nhỏ. Điều này rất hữu ích để đọc cơ chế và so sánh cục bộ giữa các nhánh, nhưng chưa đủ để khẳng định ưu thế tổng quát trên mọi miền đồ thị nhiệm vụ hoặc mọi kiểu dungeon.

Ngoài ra, matched-budget ngoại sinh với các baseline công bố rộng hơn vẫn chưa phải là phần trung tâm của bộ kết quả hiện có. Nói cách khác, luận văn hiện có bằng chứng mạnh hơn cho tính khả thi của kiến trúc và cho tác động của từng khối, nhưng bằng chứng cạnh tranh ở mức SOTA vẫn cần một vòng benchmark rộng hơn, đặc biệt là với các bộ so sánh ngoài hệ sinh nội bộ.

Một giới hạn thực dụng khác là chi phí hậu xử lý vẫn chiếm tỉ trọng đáng kể trong một số cấu hình. Kết quả đo cho thấy việc rút ngắn số bước lấy mẫu không tự động làm giảm thời gian end-to-end nếu repair và overlay vẫn phải làm nhiều việc. Do đó, các kết luận về tốc độ trong luận văn này luôn phải đọc cùng toàn bộ pipeline, không nên tách sampler ra khỏi chuỗi xử lý cuối.

5.3 Hướng phát triển

Hướng phát triển gần nhất là hoàn thiện bộ benchmark theo chuẩn matched-budget bên ngoài, đặc biệt là ánh xạ sang các bài toán và công cụ chuẩn trong pcg_benchmark để có thể so sánh trực tiếp với các baseline công bố rộng hơn. Ở tầng mission graph, cần chạy đầy đủ RANDOM, ES, GA, MAP_ELITES và FULL trên cùng ngân sách; ở tầng phòng, cần mở rộng so sánh với các baseline sinh/phục hồi phòng để tách rõ đóng góp của khuếch tán trong latent và sửa lỗi ký hiệu.

Một hướng khác là mở rộng không gian mô tả và archive cho quality-diversity, đặc

biệt là các descriptor liên quan đến độ sâu tiến trình, phân nhánh hữu ích, độ dài đường đi và độ dư tài nguyên. Khi đó, QD không chỉ đóng vai trò đánh giá hậu kỳ mà có thể trở thành mục tiêu tối ưu trực tiếp cho một nhánh của pipeline, nhất là khi kết hợp với CVT/CMA-emitter ở khối sinh mission graph.

Về mặt nghiên cứu người dùng và đánh giá chất lượng chơi, luận văn nên được nối tiếp bằng quan sát người chơi hoặc ít nhất là các proxy hành vi mạnh hơn so với pass/fail của solver. Điều này đặc biệt quan trọng vì một dungeon chơi được chưa chắc đã đủ tốt về nhịp độ, cảm giác khám phá hoặc độ phong phú của trải nghiệm.

Cuối cùng, nếu tiếp tục phát triển thành hệ thống hoàn chỉnh hơn, nên chuẩn hóa lại toàn bộ luồng báo cáo và sinh bảng thống kê để các kết quả ablation, matched-budget và OOD được tạo tự động từ cùng một pipeline. Khi đó, luận văn không chỉ có một kiến trúc tốt hơn mà còn có một quy trình thực nghiệm dễ tái lập và dễ cập nhật hơn trong các vòng nghiên cứu tiếp theo.

PHỤ LỤC

Phụ lục được tổ chức theo bốn phần theo đúng mạch đọc của luận văn: khung ký hiệu và mô hình chi phí, các thuật toán hỗ trợ, chứng minh độ phức tạp, và các phép tính minh họa. Trong phần thuật toán hỗ trợ, hai thuật toán đầu xử lý dữ liệu và kiểm tra đầu vào; bốn thuật toán sau mô tả các khối hậu xử lý, điều phối và đánh giá còn lại của pipeline.

A Khung ký hiệu và mô hình chi phí

A.1 Ký hiệu sử dụng trong phụ lục

Ngoài các ký hiệu đã nêu ở Chương 3, phụ lục dùng thêm các đại lượng dưới đây để tường minh hóa phép đếm chi phí:

$$C_{\text{enc}}, C_{\text{diff}}, C_{\text{logic}}, C_{\text{batch}}, C_{\text{teacher}}, C_{\text{archive}}, C_{\text{validate}}. \quad (1)$$

Mỗi đại lượng biểu thị chi phí thời gian của một khối chức năng tương ứng trong một lần gọi ở mức trừu tượng thuật toán.

A.2 Mô hình chi phí chuẩn hóa

Phân tích trong phụ lục dùng mô hình chi phí chuẩn hóa theo số phép toán sơ cấp và số lần gọi khối xử lý. Mô hình này phục vụ so sánh thứ bậc tăng trưởng giữa các thuật toán, không thay thế đo thời gian chạy trên phần cứng cụ thể.

B Thuật toán hỗ trợ trong phụ lục

Phần này ghi lại các thuật toán phụ trợ được rút gọn từ phần phương pháp chính, chủ yếu gồm các bước tiền xử lý, chia tập và kiểm tra hợp lệ-khả giải. Các thuật toán cốt lõi dựa trên tìm kiếm đã được mô tả ở Chương 3; phụ lục chỉ giữ lại các bước hỗ trợ thường

đi kèm để tiện tra cứu.

B.1 Tiền xử lý dữ liệu và chia tập theo nguồn

Thuật toán 8 Tiền xử lý dữ liệu và chia tập theo nguồn

Đầu vào: Tập dữ liệu thô $\mathcal{D}_{\text{raw}}$, tỉ lệ $(\rho_{\text{train}}, \rho_{\text{val}}, \rho_{\text{test}})$, seed s

Đầu ra: Tập đã chuẩn hóa $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$ và ánh xạ nguồn-split

- 1: Chuẩn hóa kích thước phòng về 16×11
 - 2: Ánh xạ tile id vào từ vựng 44 lớp
 - 3: Chuẩn hóa thứ tự nút đồ thị nhiệm vụ
 - 4: Gom các mẫu theo $\text{src}(\cdot)$
 - 5: Xáo trộn thứ tự các nguồn bằng seed s
 - 6: **Với mỗi** nguồn u **thực hiện**
 - 7: Gán toàn bộ mẫu của u vào đúng một split theo tỉ lệ đã cho
 - 8: **end Đối với mỗi**
 - 9: Loại bỏ mẫu sai schema hoặc không hữu hạn
 - 10: Đồng bộ split giữa các mẫu cùng nguồn
 - 11: **Trả về:** $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$
-

B.2 Kiểm tra hợp lệ và khả giải dungeon

Bốn thuật toán dưới đây bổ sung các khối hậu xử lý và điều phối còn lại của pipeline; chúng được giữ ở phụ lục để người đọc có thể tra cứu nhanh mà không phải quay lại toàn bộ phần phương pháp.

B.3 Sửa phòng có phản hồi ký hiệu

Thuật toán này đóng vai trò cầu nối giữa chất lượng hình học và tính đúng gameplay ở cấp phòng.

B.4 Lập lịch sinh phòng theo lớp phụ thuộc

Thuật toán này điều phối việc sinh nhiều phòng theo lớp topo và nhóm latent, nhờ đó bảo toàn các phụ thuộc logic giữa các phòng.

Thuật toán 9 Kiểm tra hợp lệ và khả giải dungeon trước khi đưa vào archive

Đầu vào: Dungeon đã ghép D , đồ thị vật lý G_{phys} , cặp start-goal (s, g)

Đầu ra: Cờ hợp lệ v , cờ khả giải u , và bộ chẩn đoán $\mathcal{I}$

- 1: $v \leftarrow 1, u \leftarrow 0, \mathcal{I} \leftarrow \emptyset$
 - 2: Kiểm tra kích thước phòng, biên ngoài, và tính nhất quán cửa-biên
 - 3: **Nếu** không thỏa **thì**
 - 4: $v \leftarrow 0$
 - 5: Ghi chẩn đoán lỗi hình học vào $\mathcal{I}$
 - 6: **Trả về:** $v, u, \mathcal{I}$
 - 7: **end Nếu**
 - 8: Kiểm tra liên thông trên G_{phys} và trên lưới ghép
 - 9: **Nếu** không liên thông **thì**
 - 10: $v \leftarrow 0$
 - 11: Ghi chẩn đoán lỗi liên thông vào $\mathcal{I}$
 - 12: **Trả về:** $v, u, \mathcal{I}$
 - 13: **end Nếu**
 - 14: Kiểm tra reachability từ s đến g
 - 15: Kiểm tra các ràng buộc khóa–cổng, vật phẩm và boss gate nếu có
 - 16: **Nếu** mọi kiểm tra đều đạt **thì**
 - 17: $u \leftarrow 1$
 - 18: **Ngược lại**
 - 19: Ghi chẩn đoán lỗi tiên trình vào $\mathcal{I}$
 - 20: **end Nếu**
 - 21: **Trả về:** $v, u, \mathcal{I}$
-

Thuật toán 10 Sửa phòng có phản hồi ký hiệu ở tầng hậu xử lý

Đầu vào: Lưới nơ-ron đầu vào x_i^{neural} , đồ thị phòng G_i , cặp start-goal (s_i, g_i)

Đầu ra: Grid cuối x_i^{final} thỏa ràng buộc biên và đường đi

- 1: $x_i^{\text{final}} \leftarrow \text{SANITIZESEMANTICIDS}(x_i^{\text{neural}})$
 - 2: $x_i^{\text{final}} \leftarrow \text{STRIPVOLATILESEMANTICS}(x_i^{\text{final}}, G_i)$
 - 3: $x_i^{\text{final}} \leftarrow \text{ENFORCEBOUNDARYSHELL}(x_i^{\text{final}}, G_i)$
 - 4: $m_i \leftarrow \text{BUILDROOMPLANTRACE}(G_i, s_i, g_i)$
 - 5: **Nếu** cơ chế sửa hậu kỳ được kích hoạt **thì**
 - 6: $x_i^{\text{cand}} \leftarrow \text{REPAIRROOMWITHFEEDBACK}(x_i^{\text{final}}, m_i)$
 - 7: **Nếu** bước sửa tạo ra cấu hình hợp lệ **thì**
 - 8: $x_i^{\text{final}} \leftarrow x_i^{\text{cand}}$
 - 9: **end Nếu**
 - 10: **end Nếu**
 - 11: $x_i^{\text{final}} \leftarrow \text{OVERLAYGRAPHMARKERS}(x_i^{\text{final}}, G_i)$
 - 12: $\text{VALIDATEROOMDIMENSIONS}(x_i^{\text{final}})$
 - 13: **Trả về:** x_i^{final}
-

Thuật toán 11 Lập lịch sinh phòng theo lớp phụ thuộc

Đầu vào: Đầu vào đã chuẩn bị (G_{phys}, C) , cấu hình sinh phòng

Đầu ra: Tập kết quả phòng $\mathcal{R}$, latent phòng và chẩn đoán batch

```

1: Nếu chế độ batch phòng độc lập được kích hoạt và  $G_{phys}$  có hướng thì
2:    $\mathcal{L} \leftarrow \text{TOPOLOGICALGENERATIONLAYERS}(G_{phys})$ 
3:   Với mỗi layer  $\ell \in \mathcal{L}$  thực hiện
4:      $\mathcal{B} \leftarrow \text{BUCKETBYLATENTSHAPE}(\ell)$ 
5:     Với mỗi bucket  $b \in \mathcal{B}$  thực hiện
6:       Nếu kích thước phòng không chuẩn (16, 11) thì
7:         Sinh tuần tự từng phòng trong  $b$ 
8:       Ngược lại
9:          $k \leftarrow \text{ESTIMATESAFEBATCHSIZE}(b, k_{\max})$ 
10:        Với mỗi chunk độc lập kích thước  $k$  trong  $b$  thực hiện
11:           $\mathcal{R}_{chunk} \leftarrow \text{GENERATEROOMBATCH}(chunk, C)$ 
12:          Cập nhật  $\mathcal{R}$  và latent đã sinh
13:        end Đối với mỗi
14:      end Nếu
15:    end Đối với mỗi
16:  end Đối với mỗi
17: Ngược lại
18:   Sinh tuần tự theo thứ tự nút ổn định
19: end Nếu
20: Trả về:  $\mathcal{R}$ , latent phòng, chẩn đoán batch

```

B.5 Sinh một phòng với cơ chế quay về mô hình giáo viên

Thuật toán 12 Sinh một phòng với cơ chế quay về mô hình giáo viên

Đầu vào: Điều kiện phòng $(z_N, z_S, z_E, z_W, b_i, p_i, G_i)$, cấu hình sampler

Đầu ra: Kết quả sinh phòng cho phòng i

```

1:  $c_i \leftarrow \text{COMPUTEROOMCONDITION}(z, b_i, p_i, G_i)$ 
2: Nếu chế độ suy luận là masked rồi rạc thì
3:    $z_i, \ell_i \leftarrow \text{MASKEDROOMSAMPLE}(c_i, G_i)$ 
4: Ngược lại Nếu chế độ suy luận là categorical thì
5:    $z_i, \ell_i \leftarrow \text{CATEGORICALCODEBOOKSAMPLE}(c_i)$ 
6: Ngược lại
7:    $z_i \leftarrow \text{DIFFUSIONSAMPLE}(c_i, G_i, \text{CFG}, \text{logic guidance})$ 
8:    $\ell_i \leftarrow \text{VQVAE.DECODE}(z_i)$ 
9: end Nếu
10:  $x_i^{\text{neural}} \leftarrow \arg \max \ell_i$ 
11:  $x_i^{\text{final}} \leftarrow \text{REPAIRANDBOUNDARYENFORCE}(x_i^{\text{neural}}, G_i)$ 
12: Nếu cần quay về mô hình giáo viên thì
13:   Sinh lại bằng mô hình giáo viên đầy đủ và vô hiệu hóa cơ chế quay về lồng nhau
14: end Nếu
15: Trả về: kết quả  $(x_i^{\text{final}}, z_i, \text{metrics})$ 

```

Thuật toán này đóng vai trò lớp an toàn cho nhánh lấy mẫu nhanh ở cấp một phòng.

B.6 Đánh giá dungeon sau ghép

Thuật toán này chốt vòng đánh giá hậu sinh ở cấp dungeon và cấp phòng.

Thuật toán 13 Đánh giá dungeon sau ghép (MAP-Elites + LogicNet)

Đầu vào: Dungeon đã ghép D , tập phòng $\mathcal{R}$, đồ thị vật lý G_{phys}

Đầu ra: Chỉ số QD, khả giải và danh sách phòng lỗi

```

1: Nếu bật MAP-Elites và thành phần MAP-Elites khả dụng thì
2:    $s \leftarrow \text{VALIDATEDUNGEON}(D)$ 
3:   Nếu  $s$  thỏa điều kiện giải được thì
4:      $\text{MAPELITESADDDUNGEON}(D, s, G_{phys})$ 
5:     Trích xuất các chỉ số QD từ  $s$ 
6:   end Nếu
7: end Nếu
8: Nếu LogicNet khả dụng thì
9:   Với mỗi phòng  $r_i \in \mathcal{R}$  có latent thực hiện
10:     $(\ell_i, info_i) \leftarrow \text{LOGICNET}(z_i)$ 
11:    Cộng dồn  $\text{grid\_reach}_i$  và ghi phòng lỗi nếu dưới ngưỡng
12:  end Đối với mỗi
13: end Nếu
14: Trả về: các chỉ số đánh giá và danh sách phòng lỗi
  
```

C Chứng minh độ phức tạp

C.1 Bổ đề phân lớp topo

Bổ đề B.1. Phân lớp topo cho đồ thị có hướng bằng phương pháp dựa trên bậc vào có chi phí

$$T_{\text{topo}}(n, m) = \Theta(n + m). \quad (2)$$

Chứng minh. Mỗi nút được đưa vào và lấy ra khỏi hàng đợi đúng một lần, nên tổng chi phí xử lý nút là $\Theta(n)$. Mỗi cạnh được duyệt đúng một lần khi giảm bậc vào của đỉnh đích, nên tổng chi phí xử lý cạnh là $\Theta(m)$. Do đó tổng chi phí là $\Theta(n + m)$ [37]. ■

C.2 Mệnh đề cho sinh cấu trúc đồ thị tiến hóa

Mệnh đề B.2 (nhánh GA). Với kích thước quần thể P và số thế hệ G , chi phí cận trên của Thuật toán 1 ở nhánh GA là

$$T_{\text{GA}} = \mathcal{O}(PG (C_{\text{exec}} + C_{\text{eval}}) + PG \log P). \quad (3)$$

Chứng minh. Ở mỗi thể hệ, phần đánh giá cá thể có chi phí tuyến tính theo số cá thể, tức $\mathcal{O}(P(C_{\text{exec}} + C_{\text{eval}}))$. Phần chọn lọc và sắp thứ tự khả thi-ưu tiên có cận trên $\mathcal{O}(P \log P)$. Nhân với G thể hệ thu được biểu thức cần chứng minh. ■

Mệnh đề B.3 (nhánh CVT-emitter). Với ngân sách đánh giá $B = P \cdot G$, chi phí cận trên là

$$T_{\text{CVT}} = \mathcal{O}(PG (C_{\text{exec}} + C_{\text{eval}})). \quad (4)$$

Chứng minh. Mỗi lượt đánh giá gọi hữu hạn các bước thực thi và chấm điểm đồ thị; tổng số lượt bị chặn trên bởi ngân sách $B = P \cdot G$. Vì vậy chi phí tổng thể tỉ lệ tuyến tính theo PG . ■

C.3 Mệnh đề cho lập lịch nhiều phòng

Mệnh đề B.4. Chi phí lập lịch của Thuật toán 4 được chặn trên bởi

$$T_{\text{sched}} = \mathcal{O}(n + m) + \mathcal{O}(N) + \sum_{b \in B} \left\lceil \frac{|b|}{k_b} \right\rceil C_{\text{batch}}(k_b). \quad (5)$$

Chứng minh. Thành phần thứ nhất đến từ phân lớp topo [37]; thành phần thứ hai đến từ duyệt và gom N phòng vào bucket theo latent shape; thành phần thứ ba đến từ số chunk thực thi trong từng bucket. Tổng ba hạng cho cận trên toàn bộ lịch sinh. ■

C.4 Mệnh đề cho huấn luyện một epoch diffusion

Mệnh đề B.5. Với $|\mathcal{D}|$ batch trong một epoch, chi phí của Thuật toán 3 có dạng

$$T_{\text{epoch}} = \sum_{j=1}^{|\mathcal{D}|} \left(C_{\text{enc}}^{(j)} + C_{\text{diff}}^{(j)} + \mathbb{I}_{\text{logic}}^{(j)} C_{\text{logic}}^{(j)} \right) + \mathcal{O}(|\mathcal{D}|). \quad (6)$$

Nếu chặn trên theo batch tệ nhất, ta thu được

$$T_{\text{epoch}} = \mathcal{O}(|\mathcal{D}| \cdot (C_{\text{enc}} + C_{\text{diff}} + \mathbb{I}_{\text{logic}} C_{\text{logic}})). \quad (7)$$

Chứng minh. Mỗi batch thực hiện đúng một vòng mã hóa điều kiện, một vòng tính loss diffusion, có thể cộng thêm logic loss, rồi bước cập nhật tham số. Cộng tuyến tính theo số batch cho ra biểu thức trên. ■

C.5 Mệnh đề cho suy luận một phòng với cơ chế quay về mô hình giáo viên

Mệnh đề B.6. Do cơ chế quay về mô hình giáo viên chỉ cho phép tối đa một lần thử lại, chi phí mỗi phòng có cận trên hữu hạn:

$$T_{\text{room}} \leq C_{\text{cond}} + C_{\text{sample}} + C_{\text{decode}} + C_{\text{repair}} + \mathbb{I}_{\text{fb}} (C_{\text{teacher}} + C_{\text{decode}} + C_{\text{repair}}). \quad (8)$$

Chứng minh. Thuật toán chỉ thực hiện một lượt sinh chính; nếu vi phạm ngưỡng thì tối đa thêm một lượt sinh lại bằng mô hình giáo viên đầy đủ. Vì không có gọi lồng nhau nên số lượt bổ sung bị chặn trên bởi 1, từ đó suy ra cận trên hữu hạn như công thức. ■

C.6 Mệnh đề cho đánh giá hậu kỳ sau ghép dungeon

Mệnh đề B.7. Chi phí của Thuật toán 7 là

$$T_{\text{post}} = \mathcal{O}(C_{\text{validate}}(D) + N_z C_{\text{logicnet}} + C_{\text{archive}}). \quad (9)$$

Chứng minh. Khối kiểm tra tổng thể gọi một lần trên dungeon đã ghép; khối chấm logic duyệt tuyến tính theo N_z phòng có latent; phần cập nhật archive được gom vào C_{archive} . Tổng ba hạng cho cận trên yêu cầu. ■

D Phép tính minh họa cho độ phức tạp thực toán

D.1 Nguyên tắc tính minh họa

Các phép tính dưới đây dùng đơn vị chi phí chuẩn hóa để minh họa cách áp dụng công thức; chúng không thay thế benchmark thời gian chạy trên phần cứng thực.

D.2 Ví dụ 1: chi phí sửa một phòng

Với kích thước phòng chuẩn $(H, W) = (16, 11)$, ta có

$$HW = 16 \times 11 = 176. \quad (10)$$

Giả sử số vòng sửa tối đa $R_{\max} = 5$ và mỗi vòng sửa tuyến tính theo số ô, tức $C_{\text{rep}}(H, W) = HW$, khi đó:

$$T_{\text{repair-room}} \leq HW + R_{\max} C_{\text{rep}}(H, W) = 176 + 5 \times 176 = 1056. \quad (11)$$

Nghĩa là cần trên minh họa tương ứng khoảng 1056 lượt xử lý ô lưới cho một phòng.

D.3 Ví dụ 2: chi phí lập lịch nhiều phòng

Giả sử dungeon có $N = 11$ phòng chuẩn, kích thước chunk an toàn $k = 8$, toàn bộ phòng thuộc một bucket chính. Khi đó số lần gọi sinh batch là:

$$\left\lceil \frac{N}{k} \right\rceil = \left\lceil \frac{11}{8} \right\rceil = 2. \quad (12)$$

Nếu đồ thị vật lý có $(n, m) = (11, 15)$, phần lập lịch topo-bucket có chi phí chuẩn hóa xấp xỉ:

$$(n + m) + N = 11 + 15 + 11 = 37, \quad (13)$$

và tổng chi phí xấp xỉ:

$$T_{\text{sched}} \approx 37 + 2 C_{\text{batch}}(8). \quad (14)$$

D.4 Ví dụ 3: ngân sách đánh giá ở tầng đồ thị nhiệm vụ

Chọn cấu hình minh họa $P = 64$, $G = 80$. Tổng số lượt đánh giá cá thể là:

$$P \cdot G = 64 \times 80 = 5120. \quad (15)$$

Với thành phần sắp thứ tự có bậc $PG \log_2 P$, ta có:

$$PG \log_2 P = 5120 \times \log_2(64) = 5120 \times 6 = 30720. \quad (16)$$

Kết quả này minh họa vì sao hạng $PG \log P$ có thể đáng kể khi P tăng.

D.5 Ví dụ 4: kỳ vọng chi phí khi có cơ chế quay về mô hình giáo viên

Đặt

$$T_0 = C_{\text{cond}} + C_{\text{sample}} + C_{\text{decode}} + C_{\text{repair}}, \quad \Delta = C_{\text{teacher}} + C_{\text{decode}} + C_{\text{repair}}. \quad (17)$$

Nếu xác suất kích hoạt fallback là p_{fb} , kỳ vọng chi phí một phòng là

$$\mathbb{E}[T_{\text{room}}] = T_0 + p_{\text{fb}} \Delta. \quad (18)$$

Ví dụ minh họa với $T_0 = 14$, $\Delta = 12$, $p_{\text{fb}} = 0.25$:

$$\mathbb{E}[T_{\text{room}}] = 14 + 0.25 \times 12 = 17. \quad (19)$$

Điều này cho thấy chi phí kỳ vọng tăng tuyến tính theo xác suất kích hoạt, nhưng đổi lại hệ thống có biên an toàn chất lượng tốt hơn.

D.6 Kiểm tra định tính nhánh latent liên tục

Để xác nhận rằng nhánh diffusion có thể sử dụng trực tiếp một checkpoint Gaussian VAE mà không làm thay đổi pipeline chính, tôi thực hiện một lần chạy smoke ngắn trên checkpoint Gaussian VAE và một lần chạy đối chứng trên checkpoint VQ-VAE cùng cấu hình. Cả hai lần chạy đều sinh ra cặp tệp `sample_0000.txt` và `sample_0000.png` trong thư mục `visual_previews` tương ứng, cho thấy hook tải checkpoint và xuất preview hoạt động đúng cho cả hai loại latent.

Về mặt quan sát định tính, preview từ Gaussian VAE thường trông đều và đối xứng

hơn vì latent liên tục cùng thành phần KL có xu hướng làm mượt các dao động cục bộ; ngược lại, preview từ VQ-VAE thường giữ lại nhiều biến thiên rời rạc hơn nên nhìn “bận” hơn. Tuy nhiên, đây chỉ là một mẫu smoke duy nhất và chỉ nên được hiểu như phép kiểm tra tương thích, không phải bằng chứng rằng một mô hình vượt trội hơn mô hình kia trong luận văn.

Tài liệu tham khảo

- [1] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, “Search-based procedural content generation: A taxonomy and survey,” *IEEE Trans. Comput. Intell. AI Games*, vol. 3, no. 3, pp. 172–186, 2011.
- [2] J. Dormans and S. Bakkes, “Generating missions and spaces for adaptable play experiences,” *IEEE Trans. Comput. Intell. AI Games*, vol. 3, no. 3, pp. 216–228, 2011.
- [3] L. T. Pereira, P. V. de Souza Prado, R. M. Lopes, and C. F. M. Toledo, “Procedural generation of dungeons’ maps and locked-door missions through an evolutionary algorithm validated with players,” *Expert Syst. Appl.*, vol. 180, p. 115009, 2021.
- [4] A. Summerville, S. Snodgrass, M. Mateas, S. Ontanon, J. Togelius, N. Wardrip-Fruin, and J. Whitehead, “Procedural content generation via machine learning (PCGML),” *IEEE Trans. Games*, vol. 10, no. 3, pp. 257–270, 2018.
- [5] A. Khalifa, P. Bontrager, S. Earle, and J. Togelius, “PCGRL: Procedural content generation via reinforcement learning,” *Proc. AAAI Conf. Artif. Intell. Interactive Digit. Entertainment*, vol. 16, no. 1, pp. 95–101, 2020.
- [6] M. Shabani, H. Aghabozorgi, H. R. Tizhoosh, and N. Miresmaeili, “HouseDiffusion: Vector floorplan generation via a diffusion model with discrete and continuous denoising,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 5466–5475, 2023.
- [7] N. Inoue, K. Kikuchi, E. Simo-Serra, M. Otani, and K. Yamaguchi, “LayoutDM: Discrete diffusion model for controllable layout generation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 10167–10176, 2023.

- [8] I. Karth and A. M. Smith, “WaveFunctionCollapse is constraint solving in the wild,” in *Proc. Int. Conf. Found. Digit. Games*, pp. 1–10, 2017.
- [9] G. Rodriguez-Torrado, M. Khalifa, J. Togelius, D. Perez-Liebana, and R. Ruiz, “Bootstrapping conditional GANs for video game level generation,” in *Proc. IEEE Conf. Games (CoG)*, pp. 41–48, 2020.
- [10] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, “Neural discrete representation learning,” in *Adv. Neural Inf. Process. Syst.*, vol. 30, pp. 6306–6315, 2017.
- [11] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 6840–6851, 2020.
- [12] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *Proc. Int. Conf. Learn. Represent.*, 2021.
- [13] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 10674–10685, 2022.
- [14] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.
- [15] Y. Benjamini and Y. Hochberg, “Controlling the false discovery rate: A practical and powerful approach to multiple testing,” *J. R. Stat. Soc. Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995.
- [16] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. Springer, 2016.
- [17] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [18] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

- [19] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, “Structured denoising diffusion models in discrete state-spaces,” *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 17981–17993, 2021.
- [20] H. Chang, H. Zhang, L. Jiang, C. Liu, and W. T. Freeman, “Maskgit: Masked generative image transformer,” *arXiv preprint arXiv:2202.04200*, 2022.
- [21] T. Park, M.-Y. Liu, T.-C. Wang, and J.-Y. Zhu, “Semantic image synthesis with spatially-adaptive normalization,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, pp. 2332–2341, 2019.
- [22] R. Hu, Z. Huang, Y. Tang, O. V. Kaick, H. Zhang, and H. Huang, “Graph2Plan: Learning floorplan generation from layout graphs,” *ACM Trans. Graph.*, vol. 39, no. 4, 2020.
- [23] L. Rampasek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, “Recipe for a general, powerful, scalable graph transformer,” in *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 14501–14515, 2022.
- [24] J.-B. Mouret and J. Clune, “Illuminating search spaces by mapping elites,” *arXiv preprint arXiv:1504.04909*, 2015.
- [25] V. Vassiliades, K. Chatzilygeroudis, and J.-B. Mouret, “Using centroidal voronoi tessellations to scale up the multidimensional archive of phenotypic elites algorithm,” *IEEE Trans. Evol. Comput.*, vol. 22, no. 4, pp. 623–630, 2018.
- [26] J. Doran and I. Parberry, “A prototype quest generator based on a structural analysis of quests from four MMORPGs,” in *Proceedings of the 2nd International Workshop on Procedural Content Generation in Games*, pp. 1–8, 2011.
- [27] V. Breault, S. Ouellet, and J. Davies, “Let CONAN tell you a story: Procedural quest generation,” *Entertainment Computing*, p. 100422, 2021.

- [28] S. Al-Nassar, A. Schaap, M. van der Zwart, M. Preuss, and M. A. Gómez-Maureira, “Questgram [qg]: Toward a mixed-initiative quest generation tool,” in *Proceedings of the 16th International Conference on the Foundations of Digital Games*, pp. 1–10, 2021.
- [29] S. Al-Nassar, A. Schaap, M. van der Zwart, M. Preuss, and M. A. Gómez-Maureira, “Questville: Procedural quest generation using NLP models,” in *Proceedings of the 18th International Conference on the Foundations of Digital Games*, pp. 1–4, 2023.
- [30] V. L. Prins, J. Prins, M. Preuss, and M. A. Gómez-Maureira, “Storyworld: Procedural quest generation rooted in variety & believability,” in *Proceedings of the 18th International Conference on the Foundations of Digital Games*, pp. 1–4, 2023.
- [31] J. K. Pugh, L. B. Soros, and K. O. Stanley, “Quality diversity: A new frontier for evolutionary computation,” *Front. Robot. AI*, vol. 3, p. 40, 2016.
- [32] M. C. Fontaine, J. Togelius, S. Nikolaidis, and A. K. Hoover, “Covariance matrix adaptation for the rapid illumination of behavior space,” in *Proc. Genet. Evol. Comput. Conf.*, pp. 94–102, 2020.
- [33] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu, “Do transformers really perform bad for graph representation?,” in *Adv. Neural Inf. Process. Syst.*, vol. 34, 2021.
- [34] C. Vignac, I. Krawczuk, A. Siraudin, B. Wang, V. Cevher, and P. Frossard, “Digress: Discrete denoising diffusion for graph generation,” in *Proc. Int. Conf. Learn. Represent.*, 2023.
- [35] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, “Consistency models,” in *Proc. Int. Conf. Mach. Learn.*, vol. 202, pp. 32211–32252, 2023.

- [36] S. Luo, Y. Tan, L. Huang, J. Li, and H. Zhao, “Latent consistency models: Synthesizing high-resolution images with few-step inference,” *arXiv preprint arXiv:2310.04378*, 2023.
- [37] A. B. Kahn, “Topological sorting of large networks,” *Commun. ACM*, vol. 5, no. 11, pp. 558–562, 1962.
- [38] K. Deb, “An efficient constraint handling method for genetic algorithms,” *Comput. Methods Appl. Mech. Eng.*, vol. 186, no. 2–4, pp. 311–338, 2000.
- [39] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv preprint arXiv:2207.12598*, 2022.
- [40] T. Hang, S. Gu, C. Li, J. Bao, D. Chen, H. Hu, X. Geng, and B. Guo, “Efficient diffusion training via Min-SNR weighting strategy,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023.
- [41] C. Holmgård, M. C. Green, A. Liapis, and J. Togelius, “Automated playtesting with procedural personas through MCTS with evolved heuristics,” *IEEE Transactions on Games*, vol. 11, no. 4, pp. 352–362, 2019.
- [42] S. Ariyurek, E. Surer, and A. Betin-Can, “Playtesting: What is beyond personas,” *IEEE Transactions on Games*, vol. 15, no. 3, pp. 348–359, 2023.
- [43] V. Rahmani and N. Pelechano, “Towards a human-like approach to path finding,” *Computers & Graphics*, vol. 102, pp. 164–174, 2022.
- [44] R. K. Dubey, S. S. Sohn, T. Thrash, C. Hölscher, A. Borrmann, and M. Kapadia, “Cognitive path planning with spatial memory distortion,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 8, pp. 3535–3549, 2023.
- [45] F. Callaway, B. van Opheusden, S. Gul, P. Das, P. M. Krueger, F. Lieder, and T. L.

Griffiths, “Rational use of cognitive resources in human planning,” *Nature Human Behaviour*, vol. 6, pp. 1112–1125, 2022.

- [46] G. L. Lancia, M. Eluchans, M. D’Alessandro, H. J. Spiers, and G. Pezzulo, “Humans account for cognitive costs when finding shortcuts: An information-theoretic analysis of navigation,” *PLOS Computational Biology*, vol. 19, no. 1, p. e1010829, 2023.